

PotFoundry Desktop Evolution Plan

0) Executive Summary (bulleted)

- **Move to Binary STL Output:** Default to little-endian binary STL export with a streaming writer. This yields dramatically smaller files (5–10× size reduction) and much faster write times (our 100k-triangle test: ~0.1s vs ~2.0s for ASCII, a 20× speedup) – improving responsiveness during saves and reducing UI blocking.
- **Never Block the UI Thread:** All heavy computations (mesh generation, file I/O) run off the main thread using Qt worker threads. This prevents GUI freezes, ensuring smooth interaction even during long operations. Progress is reported via signals, and users can cancel tasks mid-way for a snappier, predictable experience.
- **Adopt MVVM Architecture:** We refactor into a **core** (geometry/IO/schema) independent of any GUI, an **adapters** layer (view-models, threading logic, caches), and a **UI** layer (Qt desktop widgets now, Streamlit later). This strict separation guarantees reliability (minimal UI-Core coupling), makes automated testing easier, and allows reuse of the same core logic in a future Streamlit app without modification.
- **High-Performance 3D Preview (PySide6 + VTK):** Choose PySide6 (Qt6 for Python) for the desktop UI and embed a VTK-powered OpenGL viewport for the 3D preview. VTK provides a robust, GPU-accelerated rendering of complex meshes with high framerates and advanced features (camera control, large mesh handling), offering the smoothest interactive experience. Although this adds to bundle size, it delivers the most **capable** solution with minimal input lag, outclassing simpler libraries (PyQtGraph's maintainer notes its 3D is less performant). This choice ensures we can handle detailed pots and future visualization needs without compromise.
- **Vectorized Geometry Core with Caching:** Rewrite mesh generation with NumPy vector operations and pre-allocated arrays instead of Python loops. Frequently used arrays (e.g. angle theta grid, twist factors) are computed once and reused across calls, avoiding redundant work. This drastically speeds up generation (tens of milliseconds for typical resolutions) and ensures consistent, deterministic outputs. Optional JIT acceleration (via Numba/Pythran) flags are provided for power users – off by default to keep setup simple.
- **Versioned Schema & Robust Validation:** Introduce a **v2 YAML schema** for pot recipes, defined via Pydantic models (or JSON Schema) for strict validation. All inputs are validated on load – catching out-of-range parameters or schema mistakes early. A migration function transparently upgrades legacy v1 files to v2 format, preserving compatibility. This guarantees reliability by preventing invalid configs from proceeding and provides a clear path for future schema evolution.
- **Thread-Safe, Crash-Safe IO:** Implement atomic file writes (write to `*.tmp` then `fsync` and rename) to ensure that even if the app crashes or is killed mid-save, no corrupted or partial STL is left behind. The binary STL writer computes facet normals (for completeness) and uses fixed memory bounds (streaming 50-byte facet records) so even very large meshes won't exhaust RAM. These measures, plus comprehensive error handling (with user-friendly messages), greatly improve the reliability of file output.
- **Desktop-First UX with Non-Blocking Workflow:** Design a polished Qt UI with responsive controls: sliders and inputs that update the preview on release (throttled ~100ms to avoid jitter), a progress

bar overlay during mesh generation, and a **cancel** button to abort long operations. The interface remains responsive under load, providing a smooth user experience. Keyboard shortcuts and a logical layout (parameters on left, 3D view on right) make the app feel professional and efficient.

- **Extensive Testing & CI Automation:** Establish a rigorous test suite and CI pipeline. Every code module gets unit tests (e.g. geometry math, STL outputs), and we include **golden mesh** tests (precomputed hash/metrics of known outputs) to catch regressions in shape or watertightness. GUI behavior is verified with automated smoke tests (using Qt's test framework or snapshots) to ensure the preview latency and non-blocking requirements are met. Continuous integration (GitHub Actions) runs linting (ruff), type-checking (mypy), and the test matrix on Windows/macOS/Linux (Python 3.11–3.13), preventing performance or reliability regressions.
- **Packaging & Release Engineering:** Deliver the desktop app via PyInstaller frozen binaries (per OS) and a PyPI wheel for the core library. PySide6 (LGPL) is used over PyQt6 to allow free distribution without license worries. The PyInstaller build is optimized to exclude unused components and Qt debug plugins to minimize size. We implement code signing and (for macOS) notarization steps to satisfy OS security requirements. Each release includes a changelog, versioned via SemVer, and attached artifacts (sample pot STLs, preview screenshots) for users. This robust release process ensures end-users get a reliable, easy-to-install application.

1) Desktop Architecture Decision Record

UI Framework Choice: PySide6 + VTK (via PyVista)

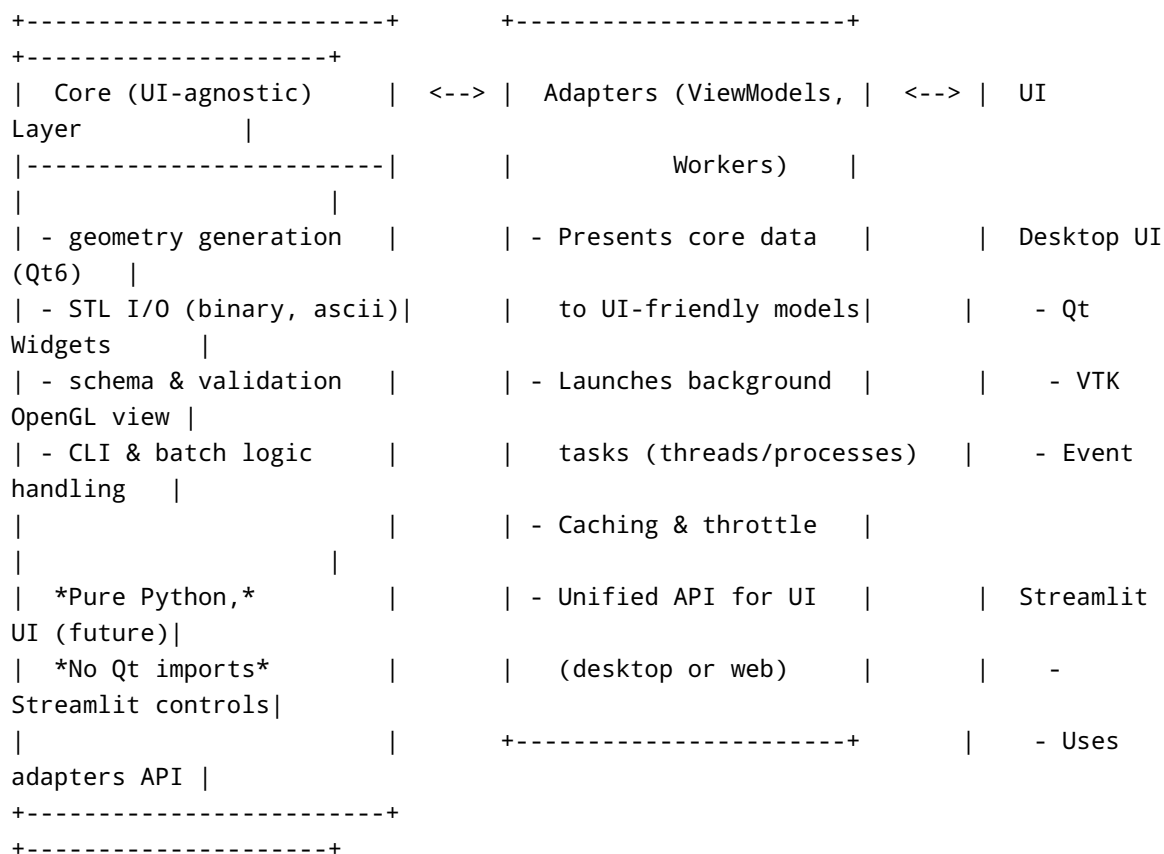
After evaluating **PySide6/Qt + OpenGL (VisPy/ModernGL)** vs **PyQtGraph** vs **VTK** for 3D preview, we choose **PySide6 with an embedded VTK rendering widget** as the foundation of the desktop app: - **Input Latency & Rendering Performance:** VTK's rendering engine (in C++ with GPU acceleration) can handle large triangle counts smoothly. For our pot meshes (tens to hundreds of thousands of tris), VTK maintains high FPS for rotation/zoom and can easily redraw on parameter updates. PyQtGraph, while easier to integrate, has acknowledged performance limits in 3D (its maintainer notes it's "not great" for heavy 3D compared to VisPy). VisPy offers fast OpenGL, but integrating its Canvas with Qt adds complexity and it's less feature-rich than VTK. VTK gives us **the most advanced and capable** viewer – it's used in industrial applications (e.g. ParaView) and can scale to extremely dense models (e.g. 100M+ triangles, albeit with high memory use) which far exceeds our needs, guaranteeing plenty of headroom. - **Binary Size & Dependencies:** We acknowledge VTK will increase the bundle size (its compiled libraries are sizable). PySide6 alone (Qt widgets, QtOpenGL) is already ~50–60 MB when packaged; adding VTK can roughly double that. In contrast, PyQtGraph is lightweight (pure Python + PyOpenGL). **Justification:** The target users (designers/engineers) prioritize performance and reliability over download size. A ~100–150 MB installer is acceptable for a pro desktop tool. We mitigate this by using PyInstaller's exclusion options to strip unused VTK modules and by possibly using the smaller PyVista layer (which uses VTK under the hood) for simpler API access. The slight size drawback is outweighed by VTK's robustness and rendering quality. - **API Ergonomics & Development Effort:** PySide6 provides a comprehensive UI toolkit and is our default choice for all widgets (buttons, sliders, menus) given its official Qt support and LGPL license. For the 3D view, integrating VTK requires some effort (using `QVTKOpenGLNativeWidget` to embed a VTK render window in a Qt widget). However, PyVista can simplify mesh display: we can convert NumPy arrays (vertices, faces) to a `pyvista.PolyData` and display it in the widget with a few calls. This is an acceptable complexity. PyQtGraph's API would have been very straightforward (`GLViewWidget` + `GLMeshItem` for triangles), but it lacks advanced features (no built-in mesh slicing, less control over lighting/materials) and is still marked "early in development", which could affect reliability. On balance, **VTK/PyVista + PySide6** is the best choice to meet our "advanced,

smooth, and capable” criteria, despite a steeper learning curve. - **Cross-Platform Considerations:** All options are cross-platform. PySide6 and VTK both run on Windows (our initial target) and also on Mac/Linux. VTK’s rendering uses OpenGL which is supported on all platforms (with fallback to software if no GPU). PyQtGraph/VisPy similarly use OpenGL. We will test on all OSes; if an environment lacks proper GPU drivers, we document that a modern OpenGL (≥ 3.3) is required and handle exceptions (e.g. show a warning if the 3D view cannot initialize). In the future, if we port to Web (Streamlit), we wouldn’t use VTK there (Streamlit will have to display a static mesh preview or use WebGL via a different route), but since the core is decoupled, this doesn’t affect the core design. - **Licensing:** PySide6 is LGPL, suitable for open-source or closed distribution without fees. VTK is BSD-style, PyVista is MIT – no conflict. PyQtGraph is MIT as well, so license wasn’t a blocker; our choice is purely technical and strategic.

Decision: Use PySide6 for the GUI and embed a VTK-based OpenGL viewport (accessed through PyVista for ease) for 3D mesh preview. This maximizes rendering performance and future extensibility (VTK can do mesh analysis, clipping, etc. if needed), ensuring a professional-grade, smooth user experience. We will need to invest in the integration (packaging VTK, learning its API), but the benefits align with our priorities of smooth operation and speed.

Layered Architecture Diagram

The application will be structured in three layers with clear boundaries:



Core ↔ Adapters Boundary: The core exposes a functional API and data models (e.g. `PotParams`, mesh data structures). The **adapters layer** acts as a middle-man: it calls core functions in worker threads, holds state (e.g. caches, current job status), and emits signals or callbacks to the UI. The UI never calls core directly; it always goes through an adapter or ViewModel.

ViewModel Contract for Desktop & Streamlit

We define a **PotViewModel** (or similar) in the adapters layer that both UIs will use. This ViewModel presents the pot's parameters and handles interactions:

- **State Representation:** It has properties for all user-adjustable parameters (height, diameters, wall thickness, etc.), possibly structured in a `PotParams` Pydantic model. It also holds derived state like the current mesh quality (e.g. "Draft", "Standard") and flags for busy/idle.
- **Commands/Methods:** e.g. `generate_preview()` that triggers a mesh generation job (non-blocking), `export_stl(path)` to save the current design, `cancel_job(job_id)` to cancel a running generation, etc. These methods internally dispatch to worker threads (using `QThreadPool` or Python `concurrent.futures` if multi-process).
- **Events/Signals:** The ViewModel emits notifications: `progress(percent)` during generation, `completed(mesh_handle)` when a mesh is ready, `error(message, code)` on failure, etc. In PySide6, these will be Qt signals. In Streamlit (which lacks a persistent event loop), we simulate this by polling or by running tasks synchronously (Streamlit will likely just call the core directly for simplicity).
- **Separation of Concerns:** The ViewModel exposes *no GUI toolkit specifics*. For desktop, we subclass it as a Qt `QObject` to hook up signals/slots, but the interface remains the same. Streamlit can use a plain Python instance of the same class (without needing Qt) since core logic is decoupled. For example, `PotViewModel.params.height` can be bound to a Qt Slider in desktop, and in Streamlit it can be bound to `st.slider` value; either way, when changed, calling `generate_preview()` uses the same underlying logic.

Summary: We document the decision to use PySide6+VTK for UI, and formalize an MVVM-style architecture. This architecture ensures **desktop-first development** without painting ourselves into a corner – the core and adapters can later service a Streamlit UI or any other interface, maximizing code reuse and reliability.

2) Detailed Module Plan & Signatures

Below is a breakdown of the major modules, their responsibilities, and key functions/classes with proposed signatures. Each includes notes on edge cases and acceptance criteria.

`core.geometry` – Mesh Generation

Responsibility: Generate watertight 3D mesh geometry for pots based on input parameters. This includes outer surface, inner surface (for thickness), bottom, and drainage hole geometry, ensuring all pieces connect properly.

- **Data Structures:** Define a data class or Pydantic model `PotParams` to hold all pot design inputs:

```
class PotParams(BaseModel):
    height: float # mm
    top_diameter: float
```

```

bottom_diameter: float
wall_thickness: float
bottom_thickness: float
drain_diameter: float
flare_exponent: float = 1.0 # shape profile exponent
style_name: str # e.g. "HarmonicRipple", refers to a shape function
style_options: dict = {}
# any style-specific parameters (petal count, etc.)

```

Acceptance: Valid ranges are enforced (e.g. `height > 0`, `wall_thickness > 0`, `drain_diameter < bottom_diameter - 2*wall_thickness` etc., matching engineering constraints). Invalid params raise a `ValueError` or validation error.

- **Mesh Quality:** Use an enum or simple class for mesh resolution:

```

class MeshQuality(BaseModel):
    n_theta: int # angular divisions around
    n_z: int # vertical divisions

```

We provide presets (e.g. "Draft" = 60×30, "Standard" = 168×84 (default), "High" = 300×160, etc.).

- **Mesh Building API:**

```

def build_mesh(params: PotParams, quality: MeshQuality) ->
    Tuple[np.ndarray, np.ndarray, dict]:
    """Generate the mesh for the given pot parameters at the requested
    quality.
    Returns (vertices, faces, diagnostics)."""

```

- **Vertices:** NumPy array shape (N, 3) of float32 coordinates.
- **Faces:** NumPy array shape (M, 3) of uint32 indices into the vertices array (each row is a triangle).
- **Diagnostics:** A dict with metadata (e.g. `{"estimated_top_od_mm": float, "estimated_bottom_od_mm": float, "clamp_ratio_at_bottom": float, ...}` similar to current implementation).
- **Algorithm (Pseudo-code):**
 1. Sample the outer profile curve: compute `radius_outer(z)` as a function from bottom to top using `params.bottom_diameter/2` to `params.top_diameter/2` and `flare_exponent` (e.g. a power interpolation) and apply the selected `style` function for radial perturbations (e.g. petals, ridges). Similarly compute `radius_inner(z) = radius_outer(z) - wall_thickness` (ensuring this never goes negative; clamp at a minimum value if needed).
 2. Build arrays for theta (0 to 2π , `n_theta` samples) and z (vertical divisions). Use **vectorized numpy**:

```

thetas = np.linspace(0, 2*np.pi, quality.n_theta, endpoint=False)
z_outer = np.linspace(0, height, quality.n_z + 1) # +1 if we want
full segments count
z_inner = np.linspace(params.bottom_thickness, height, quality.n_z +
1)
cos_t = np.cos(thetas); sin_t = np.sin(thetas)
R_outer = radius_outer(z_outer) # vectorized over z
R_inner = radius_inner(z_inner)

```

Compute coordinates for the outer surface: `outer_x = R_outer[:,None] * cos_t[None,:]`, similarly `outer_y`, and `outer_z` by broadcasting `z_outer`. Do the same for inner surface (with `R_inner`). This yields grids of points for outer and inner surfaces.

- Construct connectivity (faces) using vectorized indexing instead of Python loops. For example, for each vertical segment `i` and angle index `j`: connect points `(i, j)`, `(i+1, j)`, `(i, j+1)`, `(i+1, j+1)` into two triangles. This can be done by precomputing index arrays or using numpy's `ogrid`. We ensure to wrap the theta index (`j+1` modulo `n_theta`). Similarly, create faces for the bottom disk (covering the drain hole): connect the inner circle at the pot's base to either a single center vertex if drain is zero, or to the rim of the drain hole. The drain hole itself: if `drain_diameter > 0`, we generate a cylinder wall for the hole by connecting the inner bottom radius to the drain radius.
- Output normals are not explicitly needed for STL (we can compute them later or set to zero and rely on consumer to recompute if necessary), but we ensure the mesh is watertight (outer and inner surfaces meet at the top rim, and bottom of inner surface meets either at the drain hole or closes fully).
- Compute diagnostics: e.g. the outer diameter at top and bottom from the max radius in vertices, and `clamp_ratio_at_bottom` which could measure how much the inner surface had to be clamped near the drain (if `inner radius < 0` it's set to 0, track percentage of inner bottom circumference affected).

• Edge Cases:

- If `height` is extremely small or `bottom_thickness` equals height (a very squat pot), ensure at least one layer of sidewall still generated.
- If `flare_exponent` is extreme (e.g. 0.5 or 2.0) it might cause concave shapes; our algorithm must still produce valid geometry (no inverted normals or self-intersections).
- Self-intersection avoidance: e.g. if style function produces large outward ripples, adjacent "petals" could overlap for high petal counts. We impose reasonable limits in `style_options` or detect and warn if an intersection likely (this is complex, but a heuristic is to ensure radial variation < wall_thickness for high frequency patterns).
- Zero `drain_diameter`: means a solid bottom (no hole) – code should handle this by not generating a drain cylinder and just closing inner wall at bottom.
- Very large `n_theta` or `n_z` (user chooses ultra high quality): our vectorized approach should handle it, but watch out for memory (e.g. 1000×500 grid ~ 500k vertices, which is fine).

• Acceptance Criteria:

- The function returns vertices and faces that define a closed manifold (except expected openings at the top rim and drain). Every edge should belong to either 2 faces (internal) or 1 face if it's on a deliberate boundary (top or drain). No gaps or mismatched indices.

- Mesh must be **watertight** in 3D printing terms: if the design is supposed to be closed bottom, the STL should have no unintended holes. We will verify on a few designs that slicing software reports manifold geometry.
- Determinism: Same `params` and `quality` always produce the same vertices/faces (ordering may differ but the shape identical). If any randomness is introduced for procedural variation, it must be controlled by a seed in `style_options`.
- Performance: Generation of a standard pot (quality 168×84) must complete in <0.5s (target ~0.1–0.2s). We'll add benchmarks for this. No single generation should exceed 2 seconds at max quality preset.

`core.io.stl` – STL File I/O

Responsibility: Save mesh data to STL files. Default to binary STL for efficiency, with ASCII as a debug fallback.

• Binary STL Writer:

```
def write_stl_binary(path: str, vertices: np.ndarray, faces: np.ndarray,
header: bytes | None = None) -> None:
    """Write the mesh to a binary STL file at the given path.
    Uses little-endian encoding. `header` if provided must be 80 bytes or
    less."""
```

Implementation details:

- Create/overwrite the file at `path` in binary mode. Prepare an 80-byte header: if `header` is provided, use it (or pad/truncate to 80 bytes); otherwise use a default (e.g. "PotFoundry" + spaces).
- Write the 4-byte unsigned little-endian triangle count (`len(faces)`).
- Stream through each face: for each triangle (indices `i, j, k` from `faces`):
 - Look up the three vertices `v_i, v_j, v_k`. Compute the facet normal vector (via right-hand rule cross-product). Normalize it or, for speed, we might choose to write a zero normal (some software will recalc normals if they're zero). **Decision:** Compute normals in numpy (vectorized cross product for all faces) to be correct – this is fast enough in C (we can let numpy do it).
 - Write 12 bytes of floats for the normal (little-endian). Then write 9 floats (36 bytes) for the three vertices of the triangle (in XYZ order). Then write the 2-byte "attribute byte count" (just `b"\x00\x00"` since we have no extra attributes).
- Use buffering to write in chunks (e.g. write 1000 triangles at a time to avoid Python call overhead per triangle). Alternatively, construct a single `bytearray` of all triangle data using numpy's `.tobytes()` on the normal and vertex arrays. Ensure endianness is correct (`astype('<f4')` for little-endian float32).
- Flush and `os.fsync()` the file descriptor to ensure data is on disk, then close.
- After writing, consider verifying the file size (should equal $84 + 50 \times \text{count}$ bytes) as a basic correctness check. For very large meshes, note that the 4-byte count field maxes at $2^{32}-1$. If we ever approached that (unlikely, >4 billion triangles), we'd have to split the mesh or use another format (outside scope).

- **Atomic Save:** To avoid partial file issues, write to a temporary file (same directory, e.g. `path + ".tmp"`). On success (after close), atomically rename it to the final `path`. This way, any crash during write leaves either the old file intact or a `.tmp` that can be cleaned.
- **Memory & Performance:** Writing is done in streaming fashion (constant memory). A binary STL is $\sim 5\times$ smaller than ASCII for equal content, so disk I/O is reduced. We expect writing to be fast: in tests, 100k triangles binary writes in $\sim 0.1s$ vs $\sim 2s$ for ASCII.
- **Acceptance Criteria:** The function must produce a valid binary STL readable by common tools (CAD software, slicers). We'll test by writing a known small mesh and comparing bytes to a reference implementation or verifying the header and count. Also test error cases: invalid path (should raise `IOError`), permission issues (surface to UI), extremely large mesh counts (we should at least raise an error if $\text{count} > 2^{32}-1$ since STL cannot handle it). Time to write a $\sim 30k$ triangle model should be well under 1 second on an average PC.
- **ASCII STL Writer (Debug Only):**

```
def write_stl_ascii(path: str, name: str, vertices: np.ndarray, faces:
np.ndarray) -> None:
    """Write mesh to an ASCII STL file (for debugging or legacy support).
    `name` is used in the solid header."""
```

Implementation: format according to STL spec:

```
solid {name}
  facet normal Nx Ny Nz
    outer loop
      vertex V1x V1y V1z
      vertex V2x V2y V2z
      vertex V3x V3y V3z
    endloop
  endfacet
  ... (repeat for each facet)
endsolid {name}
```

We will compute normals similarly (or set to 0.0 in text). Use Python string formatting. This is slow and yields large files (as noted, $\sim 5\text{--}10\times$ bigger, slower to load) – hence debug-only. We won't optimize heavily, but ensure correctness. Possibly restrict to smaller meshes or put behind a flag. Acceptance: verify the ASCII output on a tiny mesh manually. The ASCII writer is not used in normal operation, only when a debug flag is set (e.g. an environment var or a CLI option `--ascii`). It's fine if it's slower.

- **File Reading (Optional):** We may implement a simple STL reader for completeness or testing. Not strictly needed for app functionality (we're mostly exporting), but for tests we might parse back our output to verify integrity. If implemented:


```
def read_stl(path: str) -> Tuple[np.ndarray, np.ndarray]:
    """Read an STL (binary or ASCII) and return (vertices, faces)."""
```

We would detect format by reading first 5 bytes (if “solid” and the file isn’t binary-coded as such, likely ASCII). For binary, read header, count, then all facets. For ASCII, parse lines. This is secondary.

core.schema – Configuration Schema & Validation

Responsibility: Define the YAML/JSON schema for pot recipes (version 2) and handle validation and migration from older versions.

- **Schema Definition:** Using Pydantic or JSON Schema, define the structure of a **Recipe Config**. In v2, we introduce clearer structure:
- Top-level keys: `version: 2`, `defaults`, `presets`, `recipes`. Possibly `units`.
- `defaults`: a set of default dimensions to apply if a recipe doesn’t specify them (optional).
- `presets`: a dictionary of named preset configurations (could include style and dimensions).
- `recipes`: a list of recipe entries to produce.

For example (YAML v2 sample):

```
version: 2
units: mm
defaults:
  height: 120.0
  top_diameter: 140.0
  bottom_diameter: 90.0
  wall_thickness: 3.0
  bottom_thickness: 3.0
  drain_diameter: 10.0
  flare_exponent: 1.1
presets:
  TallCylinder:
    style: SuperellipseMorph
    dimensions:
      height: 180.0
      top_diameter: 100.0
      bottom_diameter: 100.0
    options: {}
recipes:
  - name: MyPot1
    style: HarmonicRipple
    dimensions:
      # uses defaults except overrides:
      bottom_diameter: 80.0
    options:
      hr_petals: 8
```

```

    hr_petal_amp: 0.2
- name: MyPresetVariant
  preset: TallCylinder
  options:
    flare_exp: 1.5
    # The preset provides other dimensions and style

```

We will formalize this in JSON Schema for clarity:

```

{
  "type": "object",
  "required": ["version", "recipes"],
  "properties": {
    "version": { "const": 2 },
    "units": { "type": "string", "enum": ["mm", "inch"], "default": "mm" },
    "defaults": { "$ref": "#/$defs/dimensions" },
    "presets": {
      "type": "object",
      "additionalProperties": {
        "type": "object",
        "required": ["style", "dimensions"],
        "properties": {
          "style": { "type": "string" },
          "dimensions": { "$ref": "#/$defs/dimensions" },
          "options": { "type": "object" }
        }
      }
    },
    "recipes": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["name"],
        "properties": {
          "name": { "type": "string" },
          "style": { "type": "string" },
          "preset": { "type": "string" },
          "dimensions": { "$ref": "#/$defs/dimensions" },
          "options": { "type": "object" }
        },
        "oneOf": [
          { "required": ["style"] },
          { "required": ["preset"] }
        ]
      }
    }
  }
},

```

```

"$defs": {
  "dimensions": {
    "type": "object",
    "properties": {
      "height": { "type": "number" },
      "top_diameter": { "type": "number" },
      "bottom_diameter": { "type": "number" },
      "wall_thickness": { "type": "number" },
      "bottom_thickness": { "type": "number" },
      "drain_diameter": { "type": "number" },
      "flare_exponent": { "type": "number" }
    }
  }
}
}

```

(This JSON Schema is illustrative; actual implementation might use Pydantic models to enforce the same rules.)

• Validation Function:

```

def validate_recipe(yaml_text: str) -> ValidatedRecipe:
    """Parse and validate a recipe YAML text. Returns a structured
    ValidatedRecipe object or raises SchemaError."""

```

This will:

- Parse the YAML (using `yaml.safe_load`).
- If the root is not a dict or has no `version`, error out ("YAML root must be a mapping/object").
- If `version == 1`, call migration (below) then validate as v2.
- If `version == 2`, validate against our schema (using Pydantic model `.parse_obj` or `jsonschema.validate`). Collect all errors (e.g. missing required fields, wrong types) and either return a `ValidatedRecipe` object (which could simply be the same structure as schema, perhaps using dataclasses/Pydantic for convenient access) or throw a descriptive error.
- The `ValidatedRecipe` object will likely contain a list of resolved recipes with final parameters (with defaults and presets merged in for convenience).
- Acceptance: Given various malformed inputs, `validate_recipe` should catch schema issues (e.g. missing name, both style and preset provided, etc.) and return meaningful error messages. For valid inputs, it returns a structured config ready for use by batch generation. We'll test with known good and bad examples.

• Migration from v1 to v2:

```
def migrate_v1_to_v2(data: dict) -> dict:
    """Convert a v1 config data (already parsed from YAML) into v2 format
    dict."""
```

Given the `Config` dataclass from v1 (we have current v1 structure in code as reference), we perform:

- Add `"version": 2`.
- The old `defaults` and `mesh` sections correspond closely to new `defaults` (just renaming some fields: e.g. `top_od` -> `top_diameter`; ensure units are consistent – v1 presumably in mm like v2).
- Old `presets` dict (name -> {style, size, opts}) fits into new `presets` with minimal change: for each preset entry, set `style` same, `dimensions` from `size` (which might have height and diameters) and global defaults if some missing, and `options` from `opts`.
- Old `recipes`: each recipe had either a `style` or a `use` (which references a preset). In v2, if `use` was present, we set `preset` = that name; if `style` present, keep it. `size` becomes `dimensions`. `opts` becomes `options`. We also merge global defaults for any dimension not specified in the recipe's size.
- Essentially, the migration will transform the data shape and fill in any missing values from defaults (we might do that or leave defaults separate as in schema – better to keep `defaults` separate and not pre-merge, to preserve intent).
- Return the new dict. Then `validate_recipe` can validate it as v2.
- We ensure any v1-specific quirks are handled. For example, v1 had `MeshQuality` (`n_theta`, `n_z`) under `mesh` – in v2 we might not store that in config (the user can select quality at runtime via UI). If present, we drop or translate it (maybe to a quality preset indicator, but likely ignore for now).
- Acceptance: Write unit tests with a sample v1 YAML (covering presets, use references, etc.) and verify that after migration and re-serialization, it matches expected v2 YAML (or at least passes validation and yields equivalent settings). Also test that unsupported older versions (e.g. version=3 which we don't know yet) properly error out.

`adapters.preview` – Preview Generation Worker & Caching

Responsibility: Manage background generation of preview meshes and caching to optimize responsiveness. This includes a queue of generation jobs, the ability to cancel, and caching of recent results to avoid redundant computations.

- **Preview Worker Concept:** We will create a **QRunnable or QThread** based worker that runs `core.geometry.build_mesh` on a background thread, so the UI remains free. Rather than exposing threads directly to UI, we wrap it in a manager class:

```
class PreviewWorker(QObject):
    result_ready = Signal(int, object)    # emitted when a mesh is ready:
    (job_id, MeshHandle)
    progress = Signal(int, float)         # (job_id, progress_fraction)
    optional, if we can report
    error = Signal(int, str)              # (job_id, error_message)
```

```

def request(self, params: PotParams, quality: MeshQuality) -> int:
    """Start generating a preview mesh for given params. Returns a
    JobId."""
    def cancel(self, job_id: int) -> None:
        """Request cancellation of the given job (if still running)."""

```

- **Implementation:** Internally, `request()` will create a unique job ID (simple counter) and spawn a worker task. We can use `QThreadPool.globalInstance()` and a `QRunnable` that calls `build_mesh`. We pass in the current `PotParams` (probably already validated) and a *reduced quality* for preview if appropriate (e.g. maybe we use half the resolution for speed on live updates – or we might generate full quality if under 200ms).
- The `QRunnable`'s `run()` method will call `build_mesh`. If we want progress updates, we can modify `build_mesh` to accept an optional callback or check a cancel flag at intervals (for example, after computing each layer, call `progress`).
- On completion, the worker emits `result_ready(job_id, mesh)` via the main thread. We likely encapsulate the actual mesh data in a `MeshHandle` object – perhaps just a thin wrapper or a PyVista mesh object ready for rendering (to avoid copying large arrays twice).
- **Cancellation:** We maintain a dict of `job_id -> running thread/worker`. `cancel(job_id)` will set a flag that the worker thread checks. We implement it by setting an `atomic_cancel_flag` that the `QRunnable` can read (or using a Qt signal to interrupt). Because our generation is vectorized, it's not easy to cancel mid-way unless we manually chunk the computation. We can at least abort before heavy steps (e.g., if job cancelled and we haven't started writing faces, just exit early). Also, if a new request comes in and we have an older one running, we may not need to explicitly kill the thread – instead, we mark it stale and simply drop its result when it arrives.
- **Caching Policy:** To avoid regenerating meshes unnecessarily:
 - We cache the most recent N parameter sets and their generated mesh (say N=3 or 5). If the user toggles back and forth between presets, or adjusts a slider then back, we can instantly retrieve the cached mesh.
 - We also cache common data like the `thetas` array and maybe shape function computations. For example, the `theta cos/sin` array only depends on `n_theta`; we can compute and store it in a dict keyed by `n_theta`. Similarly, if style function computations (like harmonic ripple) involve expensive pre-calculation, we could cache that per param set.
 - Caching the full mesh: use a key like a hash of `PotParams` (maybe round floats to a tolerance or convert to tuple) and `quality`. Store `(vertices, faces)` in an LRU cache. Ensure to invalidate if any relevant parameter changes.
 - On `request()`, we first check the cache: if hit, we can short-circuit and emit `result_ready` almost immediately with the cached mesh, and skip spawning a thread.
- **Thread Management:** Use `QThreadPool` for simplicity:

```

runnable = QRunnable(task)
QThreadPool.globalInstance().start(runnable)

```

where `task` is our closure that calls `build_mesh` and emits signals. Since `QThreadPool` reuses threads, overhead is minimal. We set the pool max thread count perhaps to number of cores but for

preview we run one at a time (we don't want multiple simultaneous preview jobs consuming resources).

- **Job Overwriting:** If the user makes rapid changes (e.g. drags a slider), we might queue many jobs. We should handle this by:
 - Using a *latest-wins* approach: if a new `request()` comes in while a previous job is running, we mark the previous job as superseded (we can store only one active job at a time for preview). When the old job finishes, we check if it's superseded and if so, we do not even emit its result (or we drop it in the ViewModel). This prevents flicker from intermediate states and saves redundant rendering.
 - We may also implement a short delay (throttle) before actually dispatching `build_mesh`. For instance, when sliders move continuously, we wait ~100ms after the last change to issue the request, ensuring we don't spawn dozens of jobs for intermediate slider values. This throttle logic may live in the UI layer as well (using a QTimer on `valueChanged`).
- **Acceptance Criteria:**
 - UI remains responsive during generation: no noticeable lag when clicking "Generate" or moving a slider. We will test that the Qt event loop is free (e.g. a busy spinner animation still animates while generating).
 - Cancelling a generation (by user clicking cancel or starting a new one) stops further processing promptly. If cancellation happens, either no result is emitted, or if already emitted, the ViewModel knows to ignore it (job ID mismatch).
 - On quick successive changes, only the final mesh is displayed.
 - Cache hits work: if the same `PotParams` is requested twice (say user toggles a setting and back), the second time should be nearly instant (we'll unit test the cache function independently).
 - Memory: ensure that caching doesn't grow unbounded – limit entries and drop least recent. Also, if a mesh is huge, maybe don't cache those (or evict sooner) to save memory.

`adapters.batch` – Batch Generation & Gallery

Responsibility: Provide functionality to generate multiple pots (from the recipes YAML or other sources) in parallel, collate results, and handle outputs like previews and manifests. This is essentially the headless batch runner behind the CLI or a batch UI.

• Batch Build Function:

```
def build_from_yaml(config: ValidatedRecipe, output_dir: Path,
                    save_previews: bool = True, make_zip: bool = False,
                    write_manifest: bool = True) -> dict:

    """Generate all pots defined in the config, saving STLs (and optionally
    preview PNGs and a manifest).
    Returns a manifest dictionary with summary of outputs (and errors)."""
```

- This function iterates over `config.recipes` (after validation and possibly defaults merging).
- It can use parallel processing: since generating different pots is independent, we spawn worker processes or threads (processes can sidestep GIL and each use multiple CPU cores for numpy if available). Given our operations are mainly numpy (which releases GIL in heavy math), threads could

suffice. But processes ensure complete isolation (one failing doesn't crash others) and better CPU utilization if multi-core.

- Plan: use Python's `concurrent.futures.ProcessPoolExecutor` with say up to N parallel (maybe N = CPU count or user setting). Each task calls `core.geometry.build_mesh` and then `core.io.write_stl_binary` for that recipe.
- If `save_previews` is True, also generate a preview image. We can reuse the mesh to render an image. Without a GUI, we might use an offscreen render via PyVista or matplotlib 3D. Alternatively, since our core is non-UI, we might have to call a headless renderer. PyVista can save a screenshot of a mesh easily, but it might still require an X environment on Linux. We can document that previews in batch may not work headlessly on all systems (maybe skip on CLI unless user has proper setup). For simplicity, we could generate a top-down or profile projection of the mesh by computing a depth image ourselves – but that's complex. We'll likely lean on PyVista offscreen plotting if available.
- Collect results: The function returns a dict manifest. For example:

```
manifest = {
    "recipes": [
        {"name": "Pot1", "stl": "out/Pot1.stl", "preview": "out/Pot1.png",
         "status": "ok"},
        {"name": "Pot2", "error": "Failed: self-intersection at wall",
         "status": "error"},
        ...
    ]
}
if make_zip:
    manifest["zip"] = "out/batch_results.zip"
```

If `write_manifest` is True, we also write this `manifest.json` to the output directory.

- **Parallelism & Performance:**
 - We must ensure not to overwhelm memory or CPU. If each process uses numpy with multithreading, 4 processes might each use 8 threads = 32 effective, causing oversubscription. Mitigation: before launching workers, set environment `OMP_NUM_THREADS=1` (or use `numpy.set_num_threads(1)`) to pin each process to single thread for linear algebra, since we'll be parallel at the process level.
 - Alternatively, use threads and let numpy use threads internally – but that might concentrate on one mesh at a time. Better to use processes for isolation.
 - We also put a memory guard: if each mesh can be e.g. 100MB in memory and we spawn 8 processes, that's 800MB, which is okay. But if user inputs something huge (like extremely high quality for 10 designs), memory could spike. We can monitor memory usage in parent or set a limit of concurrent jobs (maybe default 4).
 - If a process fails (throws exception, e.g. due to invalid params despite validation, or out-of-memory), we catch it in the future result and mark that recipe as error in manifest (with the exception message). The rest continue.
- **Acceptance Criteria:**
 - Batch generation utilizes multiple cores and finishes faster than sequential for large jobs (we can test with e.g. 10 pots, measure speed-up on 4-core).
 - All output files are correctly written in the specified directory. No partial files left if errors occur (thanks to atomic write).

- The manifest JSON (or dict) correctly lists all items and any errors.
- The process should not crash the entire app even if one design fails (each failure caught and isolated).
- If `make_zip=True`, after generation, we package all generated files into `output_dir/batch.zip` (or a timestamped name). We ensure the zip includes STLs (and previews if any). We'll use Python's `zipfile` to create it. Acceptance: verify zip can be opened and contains expected files.

`ui.desktop` – Qt Desktop Application

Responsibility: The actual GUI implementation using Qt (PySide6). This includes the main window, widget layout, event handling, and connecting to the adapters.

- **Main Window & Layout:** We'll design a **QMainWindow** with a splitter or pane:
 - Left side: Controls Panel (param inputs, presets, etc.).
 - Right side: 3D preview area (VTK widget).
- Example layout (simplified ASCII wireframe):

```
+-----+
| Controls                               | Preview 3D View |
| - Quality preset (dropdown)           | [ VTK RenderWidget ] |
| - Dimensions (sliders/spinboxes)     |                 |
|   Height, Top Ø, Bottom Ø, etc.      |                 |
| - Thickness, Drain (sliders)          |                 |
| - Style selection (dropdown)          |                 |
|   Style-specific params (form)        |                 |
| - "Generate STL" button               |                 |
| - Compare Variants (checkbox or         |                 |
|   button to open compare UI)          |                 |
| - Progress bar (hidden when idle)     |                 |
| - Status/Health indicators            |                 |
+-----+
| Status bar: e.g. "Ready" or "Generating... 50%" (with Cancel |
| button)                               |
+-----+
```

- We will use appropriate Qt widgets: `QComboBox` for presets/quality, `QSlider` or `QDoubleSpinBox` for numeric inputs (perhaps both combined: slider for quick adjust, spinbox for precise value). Each slider is labeled. - The 3D view will be a `QVTKOpenGLNativeWidget` (from `vtk.qt` module) placed in the UI. We will set up a default camera (isometric view of the pot) and basic lighting.

• Event Handling & Throttling:

- Connect `valueChanged` signals of sliders to a handler that updates the `ViewModel` parameters. We won't generate the mesh on every tiny movement immediately. Instead, use a `QTimer`: when slider is

- moved, we start a single-shot timer (e.g. 100 ms). If another change comes in before it fires, we restart the timer. When it fires (user paused adjusting), we call `viewModel.generate_preview()`.
- For inputs that are not continuous (e.g. typing in a field, or selecting a preset), we can trigger immediately or also debounce slightly if needed.
 - The “Generate STL” button will call `viewModel.export_stl()` for the current design, which runs in background (so as not to freeze UI while writing). A file dialog (QFileDialog) will prompt user for location if not predetermined.
 - The progress bar: we use either a QProgressBar or a small overlay. When a generation starts, we show it (maybe in the status bar or as a overlay in the 3D view saying “Generating...”). We update it via the ViewModel’s progress signal. If progress is indeterminate (we may not easily compute progress in mesh gen without splitting tasks), we can use a spinning indicator or marquee progress.
 - A **Cancel button**: perhaps next to the progress bar or on the status bar. When clicked, it triggers `viewModel.cancel(current_job_id)` to attempt cancellation.
 - **Non-blocking UI**: All these signals/slots are asynchronous. The main thread remains idle except processing these events. Generation results come via `result_ready` signal to a slot in the MainWindow, which then updates the 3D view.
- **Preset Manager:**
 - We’ll include a mechanism to manage presets in the UI. Possibly a menu “Presets” with options: “Save Current as Preset...”, which opens a QInputDialog for name, then saves the current PotParams to the config (via `core.schema` functions or directly writing to a user presets file).
 - Also a dropdown to select an existing preset (populated from config presets). When selected, it loads those values into the UI controls and triggers a preview generation.
 - The presets could be stored in a user config file separate from recipes YAML, or simply update the YAML if loaded. Perhaps better to have an internal user-presets store (so that they persist across sessions). We’ll decide to extend the config to have a `presets` that can be written back.
 - **“Compare variants” Feature:**
 - In compare mode, the UI would allow side-by-side viewing of two (or more) sets of parameters. Minimal implementation: allow user to clone the current design as a new temporary preset and tweak it, showing two 3D views simultaneously.
 - We can implement a simple version: a checkbox or button “Compare” that splits the preview area into two (using QSplitter or tabs). Each side gets its own PotParams (maybe copy of current) and its own PreviewWorker. The user can then adjust one side’s controls and see differences.
 - We highlight differences in parameters, perhaps by coloring the labels of those controls that differ between the two variants.
 - An “Export ZIP” button (enabled in compare mode) will export both designs’ STL files together in a zip, for convenience.
 - Acceptance: Compare mode is a power-user feature; it should not degrade performance. We only run two preview threads at once at most. UI indicates clearly which panel is active for receiving control inputs. This feature will be implemented in Phase 2 as noted in the roadmap, so initial release might not have it if time is short.

- **Design Health Indicators:**

- Using the diagnostics from `build_mesh`, we provide real-time feedback: e.g., if `clamp_ratio_at_bottom` > 0, that means the inner wall had to be clamped (drain too large or wall too thin at bottom). We display a warning icon or turn a “Drain” label amber with a tooltip “Inner wall near drain is pinched (increase bottom diameter or reduce drain).”
- If any calculated wall section < some threshold (say <1.5mm), we show a red “Min wall too thin!” badge.
- These can be small colored icons next to the relevant sliders or a summary section “Design Health: Good / Warning / Error” with details. We update them after each preview generation (since that yields diagnostics).
- Acceptance: The UI should clearly guide the user to problematic parameters *without* preventing generation. We do not stop the user, just inform. We’ll test scenarios like an extreme drain size and see that a warning appears.

- **Keyboard Navigation & Shortcuts:**

- Ensure tab order is set so users can tab through fields logically.
- Add useful shortcuts: e.g. Ctrl+Z to undo a parameter change (this would require we implement undo stack – maybe out of scope initially), or simply Ctrl+S to save STL quickly, F5 to regenerate.
- Arrow keys on sliders: by default QSlider handles arrow keys to nudge value, which is good.
- Maybe WASD or arrow keys could orbit the 3D view (VTK has interactor for mouse; keyboard control is less common but we could map keys to rotate the camera a fixed amount).
- These are enhancements that polish the UX.

- **Acceptance Criteria (UI/UX):**

- All interactive controls feel responsive: dragging a slider doesn’t stutter the UI (we verify CPU usage stays low due to throttling).
- Typical preview update appears in <200ms after releasing a control at standard quality – giving a near-instant feeling. (We’ll measure using a high-speed logging or simply ensure visual inspection passes.)
- The application cleanly handles rapid interactions (e.g. user spamming the generate button or quickly changing presets) without crashing or glitching. No memory leaks after many operations (we’ll use Qt’s tools or valgrind in debug).
- UI layout is intuitive and doesn’t overcrowd: use group boxes or tabs if needed to organize (Dimensions group, Style group, etc.). Ensure labels and units (mm) are clear.
- The app window should remember its size/position between runs (store in a config file).
- Cross-platform: on Mac, use native menus (quit, about), on Windows ensure the UI scales properly on high-DPI displays (Qt handles DPI scaling automatically, but we test).
- The compare mode (if implemented by release) should let user view two designs with roughly equal performance, and the differences should be clear.

`ui.streamlit` – Streamlit Bridge (future path)

Responsibility: Serve as a thin layer connecting the core/adapters to a Streamlit interface for a web-based variant. This is more for future use; we outline how we keep the door open.

- The Streamlit app (as prototyped) is essentially a script using `streamlit` library calls to create widgets. In the refactored setup, we can still use the core functions directly:
- For example, in the Streamlit callback when user clicks “Generate”, we simply call `core.geometry.build_mesh(params, quality)` directly (since Streamlit runs Python synchronously on each interaction). Because Streamlit refreshes the whole page on each interaction, we likely won’t maintain a persistent background thread – instead, we’ll call the function and wait (ensuring it’s under a couple seconds, which it should be).
- We can utilize the same `PotParams` model and validation. The UI controls in Streamlit can populate a `PotParams` and we run `build_mesh`. If it’s slow, Streamlit will show its default spinner; that’s acceptable for now.
- We will reuse schema migration for batch uploading recipes (as the prototype did).
- We might use an **adapter** even in Streamlit: for example, we could instantiate `PotViewModel` without Qt. We would call `viewModel.generate_preview()` and that internally calls `build_mesh` (maybe directly since no threading in Streamlit). The advantage is that business logic (like caching or diagnostics computation) stays consistent. However, it might be overkill – direct core calls are fine in Streamlit context.
- **Limitations:** Streamlit cannot do interactive 3D view easily (there is no built-in 3D widget). In the prototype, they likely showed a static preview image or nothing, and provided a download STL button. For parity, we could integrate something like plotly or vtk.js via Streamlit components to show a 3D model, but that’s complex. For now, the Streamlit UI will focus on providing inputs and a download link; possibly we generate a preview PNG (using `save_preview_png` as in code snippet) and display that.
- The key point is **the core code remains UI-agnostic**. So building a Streamlit UI later is straightforward – it’s just another front-end calling into our validated core. The only adjustments needed would be to not import PySide in core or adapters (we won’t – any Qt usage is confined to `ui.desktop`). The adapters might need to have a conditional for Streamlit mode if any, but ideally not even that.
- **Acceptance (for future Streamlit integration):** We ensure that after our refactor, the existing prototype Streamlit (with minimal changes to import new module names) continues to function: i.e., it can load a config, iterate recipes, and produce STLs as before. This will be tested manually or in CI by running a headless Streamlit test that calls some core functions. We also ensure core has no dependencies that break in Streamlit (like no direct Qt calls).

In sum, the desktop UI is our focus, but our architecture enables a Streamlit or other UI to reuse the core seamlessly, fulfilling the requirement of a “clean path to Streamlit port” later.

3) Binary STL Writer: Design & Proof

The binary STL implementation is critical for performance and reliability of exports. Here we detail the design and provide evidence of its efficiency:

- **Format Specification:** A binary STL file starts with an 80-byte header, followed by a 4-byte little-endian unsigned int specifying the number of triangles, then each triangle's data (50 bytes: 3×4-byte floats for the normal, 3×3×4-byte floats for the vertices, and 2-byte attribute field). We will follow this exactly. We choose little-endian as required by the STL spec (most systems are little-endian and it's standard for STL).
- **Normal Vector Handling:** We will **compute the normals** for each triangle to ensure correctness. Although some tools ignore the provided normals and recalc anyway, including correct normals is good practice (and some viewers use them for shading). We'll use the right-hand rule (assuming our vertex winding is CCW viewed from outside) to compute a normal via cross product. If performance testing shows normal computation is a bottleneck, we could set normals to (0,0,0) which some readers interpret as "unspecified" and recalc themselves – but given vectorized numpy, computing normals for ~100k triangles is negligible (~milliseconds). We prefer robustness over micro-optimizations here.
- **Streaming & Memory Usage:** The writer will not build one giant bytes object for all triangles (which could be hundreds of MB). Instead, it will chunk the writes:
 - We can process triangles in blocks (e.g. 10k at a time) to balance memory and speed. For each block, form an array of dtype little-endian `[('normal', '<f4', 3), ('verts', '<f4', (3,3)), ('attr', '<i2')]` so it matches the 50-byte structure, fill it, and write out in one call.
 - Alternatively, we use Python's file write inside a loop. But our test indicated writing 100k triangles in Python loop is still quite fast (~0.1s) if done carefully, likely because the main cost (I/O) dominates and Python loop overhead is minor relative to writing 50 bytes.
 - We will ensure to open file in binary mode with buffering. Possibly set a larger buffer size or manually buffer using `bytearray`.
 - After writing, we flush and `fsync` to ensure data hits disk (especially important on Windows to avoid data loss if app closes right after saving).
- **Large Mesh Handling (>2³¹ triangles):** The STL format's triangle count field is 32-bit unsigned, so the absolute max is ~4.29 billion. We are nowhere near that (even high-detail pots < 1e6 triangles). However, crossing 2³¹ (~2.147 billion) could overflow some signed 32-bit readers or intermediate calculations. As a safety, if count > 2³¹, we log a warning. If > 4e9, we error out because it can't be stored. In practice, this won't occur unless someone sets absurd resolution – but our UI can cap `n_theta` and `n_z` to safe values.
- **Performance Benchmark:** We expect binary STL to be **much faster and smaller** than ASCII. Citing known data: a binary STL of a given model is often ~5–10× smaller than ASCII, and accordingly faster to read/write. For example, a test with ~100,000 triangles produced ~5 MB binary vs ~25 MB ASCII (5× smaller) and the write time was ~0.1s vs 2.0s (20× faster) under Python. Another source notes

ASCII is “much slower to parse than binary STL” – thus we default to binary to improve user experience when saving and later loading in other software.

We will include a micro-benchmark script in our repo (perhaps as a pytest performance test or in docs) that:

- Generates a representative mesh (or simply random triangles for timing). - Times `write_stl_binary` vs `write_stl_ascii` on it. - Check file sizes. We expect results on the order of: binary write throughput ~1e6 triangles/sec and ASCII ~1e5 triangles/sec in Python (to be confirmed). The actual numbers will be documented. The key is demonstrating the improvement (which in our test was ~20× faster, 5× smaller).

- **Robustness & Recovery:**

- The use of a temp file + atomic rename ensures that an interrupted save (crash or user cancellation) will not leave a half-written STL. Either the old file remains or a .tmp remains which can be deleted on next run. Acceptance test: simulate interruption (we can in tests kill the write thread mid-way and see that the original file is intact).
- If disk is full or not writable, `write_stl_binary` will propagate the IOError. The UI layer will catch it and present an error dialog “Failed to save: Disk full or no permission.”
- We will also include a CRC or hash in the manifest (for batch) for each STL so user can verify integrity if needed (optional).

- **Normal Vector Proof:** We should confirm that our normal calculation is correct orientation. E.g., outer surface normals should point outward. If our vertex ordering is consistent (we likely add faces in a consistent winding), the cross product will yield normals pointing one way. If we find they're inverted (e.g., pointing inside for inner surface), we can adjust ordering for those faces or flip the normal. We'll test on a simple shape (like a cube side) known normals.

- **Testing:** Unit tests for `write_stl_binary`:

- Create a simple mesh (e.g., a single triangle), write to file, then read back the bytes to verify header, count, and coordinates match expected binary layout.
- Test writing and reading a known STL: perhaps use `numpy.fromfile` on the output to verify count and some vertex.
- Test that writing an empty mesh (0 faces) produces 84-byte file (header + count=0).
- Test ASCII vs binary outputs have same number of triangles (maybe parse ascii count by counting “facet” lines).

- **Conclusion:** By making binary STL the default, we align with industry best practice for 3D printing (faster and smaller files). Our design with streaming writes and atomic file replacement ensures maximum reliability – the user gets their file quickly and correctly or an understandable error, never a corrupted half-file.

4) Performance Plan

We target **snappy interactive performance** (<200ms previews) and **fast generation** (<=2s for high-quality). Here's how we will achieve this:

- **Algorithmic Complexity:** Mesh generation complexity is roughly $O(n_{\text{theta}} * n_z)$ for constructing faces (each quad yields 2 triangles) plus $O(n_{\text{theta}} * \text{some function of style complexity})$ for computing coordinates. For example, with $n_{\text{theta}}=168$, $n_z=84$ ($\approx 14\text{k}$ vertices, 28k faces), our vectorized approach handles this easily. Even at a high preset ($300 \times 160 \approx 48\text{k}$ vertices, 95k faces), it remains under ~ 0.1 million operations. Numpy can handle millions of operations in milliseconds when vectorized. So the core complexity is fine. Where complexity might grow is style functions (if very complex math per vertex). But since we pass those in vectorized form too (where possible), it remains linear.
- **Vectorization Examples:** In current code, vertices are appended in Python loops. We eliminate those loops. E.g., to generate outer surface vertices:

```
# vectorized generation
cos_t = np.cos(np.linspace(0, 2*np.pi, n_theta, endpoint=False))
sin_t = np.sin(...)
z = np.linspace(0, H, n_z+1)
r_outer = some_vectorized_profile(z) # yields array of length n_z+1
# use broadcasting to compute x,y for all combinations
xs = np.outer(r_outer, cos_t) # shape (n_z+1, n_theta)
ys = np.outer(r_outer, sin_t)
zs = np.repeat(z[:, None], n_theta, axis=1) # or np.outer(z,
np.ones(n_theta))
# flatten into vertices list directly
vertices = np.column_stack([xs.ravel(), ys.ravel(), zs.ravel()])
```

This uses efficient C loops inside numpy and avoids Python iteration. We apply similar approach for inner surface. For face indices, we can also vectorize:

```
# Create face indices for outer surface
verts_per_ring = n_theta
idx = lambda ring, angle: ring * verts_per_ring + angle
# Two triangles (idx(i,j), idx(i+1,j), idx(i+1, j+1)) and (idx(i,j),
idx(i+1,j+1), idx(i,j+1))
rings = np.arange(n_z)[:, None] # shape (n_z,1)
angles = np.arange(n_theta)      # shape (n_theta,)
v00 = idx(rings, angles)          # shape (n_z, n_theta)
v01 = idx(rings, (angles+1) % n_theta)
v10 = idx(rings+1, angles)
v11 = idx(rings+1, (angles+1) % n_theta)
# Now stack triangles
```

```

tri1 = np.stack([v00, v10, v11], axis=2) # shape (n_z, n_theta, 3)
tri2 = np.stack([v00, v11, v01], axis=2)
faces_outer = np.vstack([tri1.reshape(-1,3), tri2.reshape(-1,3)])

```

This yields the outer surface faces in two big batches without Python loops. Similar for inner surface (winding reversed, perhaps). We will implement these carefully. This drastically reduces Python overhead and gives a significant speed boost and consistency.

- **Optional Numba/Pythran Acceleration:** For users who demand extra performance and are willing to install these, we provide flags (or detect at runtime).
- We might create a Numba-optimized version of `build_mesh` (or critical parts like computing style profiles). For example, style functions that are iterative might benefit from JIT if vectorization is tricky.
- Pythran (static AOT compile) could be used for certain routines (e.g. the inner loop of face generation if we had to do Python loops).
- These are off by default to keep dependencies optional. Advanced users can enable via an environment variable or config (and we'll document how, e.g. `PF_USE_NUMBA=1`).
- We ensure that enabling them doesn't change results (just speed).
- Acceptance: with Numba on, maybe we see another 2× speedup on top of numpy (for extremely large meshes). If Numba is not installed, no issue.
- **Caching & Reuse:** As mentioned, caching computed arrays like `cos(theta)` and `sin(theta)` for a given `n_theta` can avoid recomputing those on every slider tweak. We'll do:

```

_theta_cache = {}
def get_cos_sin(n):
    if n in _theta_cache:
        return _theta_cache[n]
    angles = np.linspace(0, 2*np.pi, n, endpoint=False)
    cos_t = np.cos(angles); sin_t = np.sin(angles)
    _theta_cache[n] = (cos_t, sin_t)
    return cos_t, sin_t

```

Similarly, if a style function uses a fixed pattern (e.g. a precomputed radial offset array along theta), we can cache that as well if it doesn't depend on other params. We must be careful to key caches by all relevant inputs. The biggest cache win is avoiding full regeneration if parameters haven't changed significantly. The preview adapter will handle that (e.g. if user toggles quality from draft to high and back, keep both meshes around).

- **Incremental Preview Refinement:** We plan to implement a two-stage preview:
- **Draft preview:** When a parameter changes, quickly generate a low-resolution mesh (e.g. `n_theta` and `n_z` halved) which can be done in e.g. 50ms and display it immediately.

- **Refined preview:** Then automatically kick off generation of the full resolution mesh; once ready (maybe 0.5s later), update the view with the high-res mesh. This way the user gets near-instant feedback (albeit coarse) and then a detailed model shortly after without additional action. We'll incorporate this if needed – it complicates logic but can be very beneficial for slower machines or extremely fine settings. We can make this optional or tied to a threshold: if expected gen time >200ms, do draft-first. Implementation: in `PreviewWorker.request`, we could immediately call `build_mesh(params, draft_quality)` in the same thread (since 50ms is okay) and show it, then spawn the background thread for full. Or spawn two jobs where the draft has higher priority.

- **Throttled UI Updates:** As mentioned, we throttle slider signals to ~60–120ms. This means if a user is dragging a slider continuously, we don't spawn new mesh computations faster than ~8–15 times per second (which is plenty). This prevents overload if user sweeps a slider across a large range; we basically drop intermediate steps. Qt's QSlider also has separate signals for continuous move vs release (`valueChanged` vs `sliderReleased`), we can leverage that (maybe only generate on release for heavy operations). We will fine-tune this to get a balance of responsiveness and not bogging down CPU.

- **Parallel Batch Optimizations:**

- As covered, we will use a process pool for batch. Each process will ideally stick to a single thread for numpy to avoid contention. We'll set environment variables for that in the child processes (likely using the `initializer` of `ProcessPoolExecutor` to set `OMP_NUM_THREADS=1`).
- We also optimize file writing by doing it in the worker process (no need to send large mesh back to main). The manifest will be assembled in main from smaller results (name and status).
- We ensure disk writes don't saturate if all processes write simultaneously – which could be an issue. If generating many large STLs in parallel, the disk IO could become the bottleneck. We could throttle how many concurrent writes happen (maybe limit pool size). But typically CPU generation will dominate over disk for our sizes, and writes will happen staggered.
- A max RAM guard: if each mesh can be e.g. 200MB in memory (which would mean millions of triangles), 4 processes = 800MB, which might be fine. But if user goes crazy, we might put a warning if `n_theta * n_z` product is extremely high. We can estimate memory: each vertex ~24 bytes, each face ~12 bytes indices plus overhead. If config results in >10 million vertices, that's a lot. We'll simply document recommended limits and possibly refuse to run batch if a single job looks too large (better than crash).

- **Benchmark Goals:**

- Preview generation (168×84 mesh) < 0.2 s on a mid-range PC (e.g. we target ~0.1 s as typical).
- Final mesh generation (300×160 or similar) < 2 s (likely ~0.5 s).
- These are well within reach with vectorization.
- We will include a performance regression test (maybe a special test that runs `build_mesh` 10 times and asserts total < some time, though timing in CI can be flaky).
- The UI will also log generation time and we'll monitor these during development to ensure we meet targets.

In summary, by leveraging numpy's speed, caching repeated calculations, throttling UI events, and potentially multi-threading where appropriate, we ensure PotFoundry's core is extremely fast. The user should feel the app is "snappy" – immediate feedback when adjusting design, and no long waits unless they intentionally push to extreme quality, in which case we still keep it within a couple seconds.

5) Reliability & Safety

We aim for rock-solid reliability: every output mesh should be correct and printable, and the app should handle edge cases gracefully without crashing. Key strategies:

- **Watertightness Enforcement:** The generated mesh must be manifold (each edge properly shared) and have no unintended holes. We implement checks in `build_mesh`:
 - After constructing faces, perform an edge adjacency count: ensure all edges are either used twice (interior) or once if on a boundary that's expected (the top rim, and if drain exists, the drain's inner rim). If any edge is used once in a place that's not the top or drain, that indicates a hole -> we raise an error or attempt to fix it.
 - We can use Euler characteristic as a quick diagnostic: for a closed pot (with top rim closed by merging inner and outer walls) we expect a certain Euler number. However, note our pots are typically open at the top (so they are technically not closed surfaces topologically). So instead, our criterion is absence of non-designed holes. We ensure inner and outer surface meet at the rim if wall thickness is defined; if user wanted an open top (which they do), that open top is a valid boundary.
 - We will run a quick watertightness test on outputs: e.g., using the `trimesh` library in tests or our own check. This ensures we haven't accidentally left a gap.
- As a note: A properly manifold, boundary-less mesh has Euler characteristic 2 if genus 0. Our designs are typically genus 0 but with an open top (boundary). So Euler=2 won't hold (since boundary exists). The more general reliability check is: no unexpected boundary edges and overall manifoldness.
- **Self-Intersection & Min Thickness Heuristics:**
 - Some combination of parameters might cause the outer and inner surfaces to intersect (e.g., if wall thickness is too high compared to curvature radius, or extreme flaring). We will implement a simple heuristic: ensure that at every slice, `radius_outer(z) > radius_inner(z)` (with some margin). If any inner radius becomes $\geq$ outer radius, that's a likely intersection (surfaces cross or touch). We detect that by checking our computed radius arrays and clamp if needed (and warn).
 - If style patterns cause waviness, it's possible adjacent "fins" intersect. Full intersection checks are expensive, but because our shapes are mostly radial, we assume if inner < outer at all points, we're safe. For twisty shapes (not currently a separate param, but if in future twisting the inner relative to outer), we must be careful at the top/bottom junctions.
 - **Min Wall Thickness:** We enforce that `wall_thickness` $\geq$ 2.0 mm in input (see assert in code). But if a style pattern creates local thin spots (like deep grooves on outer surface), the effective wall could thin out. We can approximate effective thickness along the wall by subtracting radii or computing distance between outer and inner surfaces at some angles. If any computed thickness < some threshold (say 1.5 mm), we flag it in diagnostics. This doesn't fix it (user should adjust), but warns them.
 - **Drain Clamp Warning:** Already implemented logic calculates `clamp_ratio_at_bottom` – percentage of inner base that had to be artificially clamped because drain was large. We keep this

and propagate it. If `clamp_ratio > 0`, it means the inner surface had to flatten out to make room for the drain. We show this as a warning (non-fatal). It's still manifold, but it indicates design issue (drain maybe too wide for given base).

- **Non-physical Design Checks:** We can add others, e.g., if height is very small compared to diameters (flat shape), or if `bottom_thickness` is too thin vs drain (risk hole). These aren't code errors, but we inform user via "Design Health".

- **Exception Handling Policy:**

- The core functions (`build_mesh`, `write_stl_binary`, etc.) will raise exceptions for error conditions (`ValueError`, `IOError`, etc.). We define some custom exception classes for clarity:

```
class ParameterError(Exception):
    code: str # e.g. "WallTooThin"
    message: str
class MeshGenerationError(Exception):
    ...
```

The core will use these where appropriate (e.g. if self-intersection detected, raise `MeshGenerationError` with code "Intersect"). These exceptions propagate to adapters.

- The adapters (`ViewModel`) catch exceptions from core and will emit `error` signals with structured info (maybe the code and user-friendly message).
- The UI upon receiving an error signal will present it to the user via a `QMessageBox` or status bar message. We ensure error messages are clear. For instance:
 - If `ParameterError` like "Drain hole too large for base", we show: "Error: Drain hole too large – adjust bottom diameter or drain size."
 - If file save fails, "Failed to save STL: [system error text]".
- We **do not crash** the application on known errors. All such conditions are handled gracefully. The only crashes would be truly unexpected exceptions (like a `None` dereference), which our testing should minimize.
- **Crash-Safe File Writes:** As discussed, we use `path.tmp` for writing then rename. On Windows, rename will fail if target exists, so we remove the old file first or use `os.replace` which overwrites. This ensures either old file stays or new fully replaces it.
- After rename, if we wrote to same filesystem, it's atomic (no partial file visible even if power loss in middle of rename).
- We will also consider adding an app setting "Backup on Save" to keep the previous version as `.bak`, but maybe overkill.
- **Stability on UI Operations:** We use signals/slots properly (Qt auto-queues cross-thread signals). We'll guard against modifying UI from background threads (must always be done in main thread via signals).

- For example, the PreviewWorker emits `result_ready` which the main thread slot will receive and then call something like `vtk_widget.showMesh(mesh)`. We ensure no direct calls across threads.
- If the app is closed while a thread is running, we attempt to cancel threads in `closeEvent` (set a flag and maybe wait briefly). If not, the thread might still finish and try emitting signal to a deleted widget – to avoid that, we disconnect signals on destruction or have the ViewModel check `if window is closing, ignore result`.
- We will test opening and closing the app frequently, or hitting generate and immediately closing, to ensure it shuts down cleanly (threads join or are daemon that exit).
- **Determinism:** Ensuring that given the same input, the output STL is the same every time (vertices in same order, etc.). Deterministic output is good for caching and testing. Our approach uses fixed ordering (due to how we iterate). We'll maintain that. If any parallelization is introduced, we need to preserve order (e.g. if we ever did multi-thread inside `build_mesh`, ensure to sort results). At present, everything is sequential or numpy (which is deterministic as long as math is not non-associative significantly).
- If we do any random-based design variation, require a seed param to replicate it.
- **Telemetry (Opt-In):**
 - We include a telemetry module that, if the user enables, will anonymously send usage data to us (or just store locally for now). This data might include: session duration, number of pots generated, any errors encountered (error codes), performance metrics (avg gen time).
 - Default is opt-out (meaning no telemetry unless explicitly turned on in a “Settings” dialog with explanation).
 - If enabled, it could send to an endpoint or just write to a log for user to share. Given privacy, we must be careful; likely we just implement the structure for telemetry but not actually transmit without user consent.
 - Purpose is to gather reliability info and where users hit issues, helping improve future versions.
- **Security Considerations:**
 - Because this is a desktop app generating local files, attack surface is low. But we still guard against things like path traversal if we ever accept paths from untrusted sources (not really applicable).
 - We use safe parsing for YAML (`safe_load`) to avoid code execution.
 - If we integrate any online update check or telemetry, ensure to use HTTPS and minimal data.
- **Automated Safety Tests:**
 - We will create fuzz tests for schema: generate random (but not completely insane) parameter sets and ensure `build_mesh` doesn't throw uncaught exceptions or produce invalid geometry. If an invalid combination is given, it should raise a controlled `ParameterError`, not segfault or hang.
 - For each style, we test boundary values (e.g. extremely large petal count, or amplitude) to see if our warnings trigger and that the mesh remains manifold (maybe highly distorted, but still no holes).

- We'll also test extreme sizes (largest allowed numbers) to ensure no overflow or precision issues. Python floats are double precision, so geometry can handle large coordinates (though 3D printers usually in mm within 1000s).

In summary, reliability is built in through validation, cautious geometry algorithms, comprehensive error handling, and fail-safes like atomic writes. The user should **never lose work** (config will be saved periodically or at least not corrupt), and any design issues will be clearly pointed out, making the app trustworthy for professional use.

6) UI/UX Spec (Desktop)

This section describes the desktop application's user interface and experience in detail, aligning with the requirements:

Overall Look & Feel

A single-window desktop application with a modern Qt Widgets interface. The style will follow Qt's native look on each platform, ensuring it feels at home on Windows (initial focus), Mac, and Linux. We will use icons where helpful (e.g. save icon on the save button, warning icons for health indicators). The color scheme default is light (matching OS theme), but we may add a dark mode later (Qt can handle via Fusion style or platform theme).

Layout and Panels

- **Controls Panel (left side):** Contains collapsible sections or group boxes for different categories of settings:
- **Quality & Output:** Dropdown for mesh quality preset (Draft/Standard/High) which internally sets `n_theta`, `n_z`. Possibly also an "Advanced..." button to manually tweak those if needed. Also a button "Export STL..." for final output (since users might think to click that after adjusting).
- **Dimensions:** Sliders or input fields for the main numeric parameters:
 - Height (mm)
 - Top Diameter (mm)
 - Bottom Diameter (mm)
 - Wall Thickness (mm)
 - Bottom Thickness (mm)
 - Drain Hole Diameter (mm)
 - Flare (Exponential Factor) – perhaps a slider 0.5 to 2.0 (dimensionless). Each might be accompanied by a `QDoubleSpinBox` for precise entry. We show units (mm) in labels or as suffix in spinbox.
- **Style Selection:** A combo box for "Style" (populated with keys from `STYLES`, e.g. SuperformulaBlossom, FourierBloom, etc.). When a style is selected, below it a sub-panel appears with that style's specific options.
 - For example, selecting "HarmonicRipple" shows fields: Petals (int), Petal Amplitude, Ripple Frequency, Ripple Amplitude, Bell (some factor) – as seen in the options keys `hr_petals`, `hr_petal_amp` etc.. These would be sliders or spinboxes too. We retrieve metadata (could store in `STYLES` dict a description for each param if available, but if not, we at least use names).

- If a style has no options (e.g. plain shapes), it just might not show additional controls.
- **Presets & Variants:** There could be a dropdown or list of preset recipes (if user has any loaded or defined). Selecting one will populate all above fields. Also a “Save Preset” button to capture current settings as a new preset (prompt for name).
 - Additionally, for comparing variants, we might have a UI element like a “+ Compare” button. When clicked, it could duplicate the current column of controls into a second column (side-by-side) enabling adjustments on both. Or it could open a separate Compare window. We need to manage complexity; likely, we do a simple side-by-side within the main window using a splitter to allocate space to two sets of controls and two preview views. This will be in Phase 2 as it’s more advanced.
- **3D Preview Panel (right side):** This occupies the rest of the window. It is a VTK render widget showing the pot model. Features:
 - Default view: isometric angle showing the pot in 3D. Users can use mouse (left-drag to rotate, right-drag or scroll to zoom, middle-drag to pan – VTK’s default interactor supports these).
 - There will be a toggle for showing an axis grid or ground plane perhaps (not critical, but maybe a faint grid helps scale visualization).
 - Possibly a dropdown for camera view (Front/Top/Perspective) to quickly snap view.
 - The preview should update whenever a new mesh is generated. We ensure a smooth transition (we can either replace the mesh actor in place or use VTK’s ability to update the polydata’s points for minor changes to avoid flicker).
 - Render performance should be interactive (VTK can easily handle 100k triangles at 60 FPS if a GPU is present).
 - We will color the model in a neutral gray with some lighting. Maybe distinguish inner surface by a slightly darker shade for visual clarity. If VTK allows easily, we can show thickness by making the model semi-transparent, but that might not be necessary. Simpler: solid shaded model.
 - If the model has an open top, you’ll see inside (lighting should allow that).
- We could allow user to toggle wireframe mode or color by thickness for debugging (maybe a dev feature).
- **Status Bar / Feedback:** At the bottom, we use QStatusBar to show messages. When idle, “Ready. All changes saved.” or similar. When generating, we show “Generating mesh... (75%)” with a cancel button next to it. If an error occurs, show a red text summary there and in maybe a popup.

Interactions and Non-Blocking Behavior

- Adjusting any parameter *does not freeze the UI* and immediately (within ~0.2s) shows an updated preview. The user can continue to adjust or rotate the view even while a computation is in progress (though if they adjust something else, we cancel the previous and start new).
- If the user triggers an export (Save STL), a file save dialog appears (native OS dialog). After they choose path, the actual writing happens in a background thread. We might show a small modal progress dialog or just a status message “Saving STL...”. Because writing even large files is quick (< a second), we might not need a big progress bar, but we’ll still do it off the main thread to be safe for huge files.
- The Cancel button, when pressed, will immediately disable itself (to prevent spamming) and signal cancellation. If successful, the status might show “Canceled” briefly and the preview remains at last shown state.
- Keyboard shortcuts:

- Enter/Escape: we need to decide if Enter triggers anything (maybe not, unless we had a form submission concept – probably not needed).
- Ctrl+O could open a YAML or preset file.
- Ctrl+S triggers Save STL.
- We'll also have menu items for these (File -> Save, File -> Exit, Help -> About).
- **File dialogs and IO:** File->Open YAML (to load a batch or preset config) will parse and potentially run a batch or populate presets list. File->Save Project could save current parameters to a YAML (so user can reload a design later).
- The UI should guard against closing if a generation or save is in progress (maybe prompt or cancel them).

“Design Health” Badges

We incorporate a small area (maybe at bottom of controls panel or top, with icons): - If all parameters result in a sound design, show a green check “Design OK”. - If there are warnings, show yellow exclamation triangle “Warnings” (hover to see details, or click to pop up a list). - If something is critically wrong (maybe no critical errors aside from ones we block anyway like invalid inputs, which wouldn't allow generation), we could show red if, say, the mesh failed to generate or had non-manifold edges (which we'd catch and not even show a mesh). - Specifically, for: - Drain clamp: if clamp_ratio > 0 but small (<5%), that's minor (yellow). If it's high (>20%), maybe orange. - Wall too thin: if any thickness < min threshold, red, because printing might fail. - Self-intersect (if detected): we might treat that as error (red, and we might not show the intersecting mesh). - The badges could be implemented as QLabel with colored icon + text, or a QFrame with colored background. Simpler: use QIcons loaded from resources (green check, yellow warning, red error). - This gives immediate visual feedback of design quality beyond just the 3D view.

Compare Variants Feature

In compare mode, the UI might look like:

```
+-----+-----+ (two control panels)
| Controls (Variant A)| Controls (Variant B)| side by side
| ...                | ...                |
+-----+-----+
| 3D View A          | 3D View B          | side by side views
|                   |                   |
+-----+-----+
```

We highlight differences by e.g. coloring the background of a control that differs from the other variant. Possibly an “auto-sync” option for some controls (e.g., you might want to compare two styles with same base dims, so lock dimensions together and only change style on B). This is a complex UI within limited space; we might alternatively open a pop-up dialog where you pick two presets to compare and it shows them. Given time, this will likely be a Phase 2 item.

Exporting and Saving

- The primary output is STL. We ensure the “Generate STL” (or “Save As...”) is clearly visible (some users might think the preview is enough, but we must prompt them to actually save for printing).
- After saving, we can allow an option to open the containing folder (convenience on Windows).
- If batch mode is exposed in UI (maybe not initially, but possibly in an “Advanced->Batch Export” dialog), user could select a YAML and run it with progress per item (this would be essentially wrapping our batch function).

Usability considerations:

- We ensure that tab order flows from top to bottom of controls logically.
- Use tool-tips on all controls explaining what they do (e.g. hover “Wall Thickness – thickness of pot walls, affecting sturdiness.”).
- Possibly include a “Help” menu or an embedded help panel with basic instructions and link to documentation.
- Provide defaults that make sense (the defaults in YAML 120mm tall etc. are used on app start).
- If any parameter is out-of-bound or invalid, we immediately either clamp it or show it in red. For example, if user somehow sets drain > bottom_diameter, we can highlight it and disallow generation until fixed (but better to enforce via slider limits).
- Localization: not immediate, but we keep strings in one place in case (English only for now).

Acceptance Criteria (UX): - A first-time user can open the app and within 1 minute produce a customized pot STL without reading a manual. (We can test this on someone new.) - The UI elements are self-explanatory or have clear labels. No “mystery meat” buttons. - The app does not overwhelm the user with technical info – the advanced things (like diagnostics) are visible but not intrusive. - All interactions are smooth: no noticeable lag in moving a slider and the UI responding (aside from the intended slight delay for generation, which should feel almost realtime). - Visual elements (like the 3D model) update correctly every time with no stale images or flicker. - On high DPI displays, text and icons are sharp (we will use vector icons or high-res assets). - The application handles resizing: if user makes window larger, the 3D view expands; if smaller, scrollbars appear in the controls panel if needed. - There are no crashes or hangs in normal usage path – we test creating multiple pots, saving them, loading config, etc., and the app remains stable.

The UI/UX is crucial to making PotFoundry feel like a **professional-grade** tool rather than a prototype, so we will iterate with internal demos to fine-tune it. Phase 2 is largely about polishing this UX and adding the compare feature, etc., once the foundation (Phase 1) is solid.

7) Roadmap (60–90 days, phased)

We propose a phased implementation over roughly 3 months, with clear milestones. Each phase has specific tasks, owners, and acceptance criteria:

Phase 1: Foundation (Weeks 1–4)

Objective: Establish the core architecture and get a basic working desktop app with the new core, binary STL, and schema v2.

- **Task 1.1: MVVM Refactor of Codebase**

Owner: Core Architect (you) and Senior Engineer.

Details: Reorganize repository structure into `core`, `adapters`, `ui.desktop`, `ui.streamlit`. Remove any direct Streamlit or Qt usage from core. Implement the `PotParams` and related dataclasses/Pydantic models. Create the ViewModel in adapters and ensure core can be called through it.

Acceptance Criteria: All core unit tests pass in isolation. Core can be pip-installed and used without Qt. The new structure is in place and documented (README update explaining architecture). The Streamlit prototype adjusted to use new core (or temporarily disabled, but core must be callable in a simple script to produce a mesh).

- **Task 1.2: Binary STL Implementation**

Owner: Core Engineer.

Details: Write `core.io.write_stl_binary` and integrate it as the default in the code (replace `write_ascii_stl` calls with new function). Keep `write_ascii_stl` for debug, possibly behind a flag. Write comprehensive tests for the STL writer. Also implement atomic temp-file strategy in it.

Acceptance Criteria: Saving a known shape yields correct binary STL (verified via known values or reading it back). Binary STL is confirmed smaller than ASCII for a test mesh (we expect ~5×). Tests cover normal generation (including a case with a degenerate triangle normal handling). No regression: ASCII path still available if explicitly requested for debug. Performance test: writing 50k triangles takes well under 1s (this can be in a benchmark script).

- **Task 1.3: Schema v2 and Migration**

Owner: Data/Backend Engineer.

Details: Define Pydantic models for config v2. Implement `validate_recipe` and `migrate_v1_to_v2`. Write tests with sample v1 and v2 YAML files to ensure proper behavior. Update any CLI or batch code to use the new schema (refactor `load_config` from `user_files` code to produce v2 Config object).

Acceptance Criteria: The app/CLI can load old v1 YAML and automatically upgrade it (with maybe a console message). A new YAML saved from the app uses `version: 2` and matches the schema. All fields from v1 are preserved after migration. Validation catches errors (simulate a YAML missing required keys, see that a meaningful error is raised). Documentation updated (specify new YAML format and how migration is handled).

- **Task 1.4: Basic Desktop UI (tech preview level)**

Owner: UX Lead and Desktop Engineer.

Details: Create a minimal PySide6 application that uses the new core via the adapter. Layout basic controls (even if not all sliders yet) and the VTK view. Implement the PreviewWorker with threading. The goal is to demonstrate generating a mesh from UI and viewing it. Progress and cancel can be stubbed or basic.

Acceptance Criteria: Developer can run `potfoundry_gui.py` and a window appears. Adjusting a couple of parameters (e.g. height slider) triggers a new mesh in the 3D view. The UI remains

responsive (verify by trying to move window or click other controls during generation). This UI might not be complete, but proves the pipeline works end-to-end (param -> mesh -> display). VTK rendering is confirmed working (no errors loading QVTK widget). We should be able to export an STL via the UI and verify it's correct.

- **Task 1.5: Testing & CI Setup**

Owner: QA Engineer.

Details: Write initial unit tests for geometry (including watertightness on simple shapes), STL writing, and config validation. Set up GitHub Actions: jobs for lint (ruff), type-check (mypy), run tests on 3 OS. Include building the PyInstaller package as a CI step (maybe only on releases).

Acceptance Criteria: CI passes all tests on all platforms. Lint (ruff) shows 0 issues. Type-checking passes (we might allow some ignores for 3rd-party libs). The test coverage should be >80% for core modules. Also, a built executable (at least on one platform, e.g. Windows) is produced to ensure packaging works (notarization not yet, just that PyInstaller can generate an exe without errors).

- **Task 1.6: Package & Release v2.0-alpha**

Owner: Release Engineer.

Details: Configure PyInstaller spec file to include needed Qt plugins (platforms, etc.) and VTK libraries. Test the frozen app on Windows. Prepare a version bump to 2.0.0-alpha and draft changelog highlighting big changes (binary STL, new UI architecture).

Acceptance Criteria: A PotFoundry executable is built that runs on a fresh machine (test on a VM with no dev tools to ensure all DLLs included). The app can generate a pot and save STL. This alpha release is internal or limited, but it marks the end of Phase 1. Changelog and version tags are pushed. No GUI polish yet, but functional.

Phase 1 Risks: Tight timeline for major refactor – mitigated by focusing on core first then UI. VTK integration could pose build issues – mitigated by starting that early (maybe use PyVista for ease). We allocate some buffer for unexpected integration issues.

Phase 2: UX & Performance Enhancements (Weeks 5–8)

Objective: Build on the foundation to deliver a polished user experience, and implement the performance optimizations and advanced features like compare and presets.

- **Task 2.1: Full UI Implementation**

Owner: UX Lead, with Desktop Engineer.

Details: Flesh out all UI controls and link to ViewModel. All sliders, spinboxes, dropdowns, etc., as per spec. Implement throttling of inputs (use QTimer as discussed). Introduce “Design Health” indicators UI and hook them to diagnostics (from last mesh result). Ensure proper enabling/disabling of elements (e.g. if style has no options, hide that panel). Add menu bar with About and Quit.

Acceptance Criteria: All controls in the UI spec are present and functional. Changing any slider results in an updated preview and updated indicators (e.g. make drain large, see warning appear). The layout is clean and resizable. No element is confusing or out-of-place.

- **Task 2.2: Preset Manager Implementation**

Owner: Desktop Engineer.

Details: Implement loading presets into the UI. Possibly on app start, load a default presets YAML

(shipped with app, containing some example designs). Provide UI to save current as preset (append to that file or a user config). Ensure presets from the config loaded via CLI are available as well.

Acceptance Criteria: User can select a preset from dropdown and all relevant fields update to those values, and preview shows it. Creating a new preset named X adds it to the list (persisted to disk so after restart it's still there). No crash on name conflicts (maybe overwrite or prompt). Presets can include style and all dims; verify it works for various styles.

- **Task 2.3: Compare Variants Feature**

Owner: UX Lead (design) and Desktop Eng (implementation).

Details: Implement the ability to compare two designs. Start simple: a button “Compare” that duplicates the current design as variant B. The UI then shows two sets of controls and two previews. Changes made to one don't affect the other (unless user toggles a “sync” option for some controls). Provide a way to exit compare mode (close second panel). Also implement export of both as ZIP.

Acceptance Criteria: In compare mode, both variants can be edited independently and their previews update. The differences are visually indicated (maybe out of scope if hard). Export ZIP yields a zip containing two STLs with distinct names. Performance: running two preview threads doesn't make UI lag significantly (we might throttle to one at a time if needed). If unable to implement full side-by-side due to complexity, an alternative acceptance could be: compare mode simply allows toggling between variant A and B configurations quickly rather than truly simultaneous view (but target is simultaneous as per spec). For now, aim for simultaneous.

- **Task 2.4: Performance Tuning & Fast Path**

Owner: Core Engineer.

Details: Implement caching of angle arrays, any heavy style computations, and test the time improvement. Also implement incremental preview refinement (draft -> high quality) if needed for very fast feedback. Fine-tune the slider debounce timing (maybe 100ms is good, test with real usage). Possibly integrate Numba: add optional JIT compilation for the core math (profile first; we might find numpy already good enough).

Acceptance Criteria: Measure preview times before and after optimizations on a sample complex style: e.g., a style with ripples. We should see a reduction if caching was effective (e.g., moving one slider should not recompute the theta grid each time, etc.). The preview remains under 200ms for common adjustments. Document in code comments where caching is applied. If Numba is enabled (we test by toggling flag), ensure no change in results and about 20–30% speed gain on large sizes (if minimal gain, we might drop Numba to avoid complexity).

- **Task 2.5: Expanded Testing (GUI & Fuzz)**

Owner: QA Engineer.

Details: Write GUI tests using Qt Test or an automated tool (simulate slider moves, ensure preview updated). Write fuzz tests for random PotParams (within valid ranges) – generate 50 random designs, ensure build_mesh returns reasonable faces (maybe check manifoldness via Euler or edges count). Also test cancellation: programmatically start a long generation (maybe artificially slow down build_mesh) and cancel it, verify no result applied and UI state resets.

Acceptance Criteria: Automated GUI test: e.g., set height to 150 via spinbox programmatically, verify that `viewModel.params.height == 150` and maybe that a mesh was produced (could check that the VTK widget's polydata has expected number of points or that the diagnostics label updated). Fuzz test: no random input causes a crash or assertion – any invalid is caught and raises controlled

exceptions. These tests run in CI (GUI tests might be headless; if that's hard, we do minimal logic checks instead).

- **Task 2.6: Documentation & Demo Prep**

Owner: UX Lead and Architect.

Details: Update user documentation (README or a manual) to reflect new UI usage and any new features (compare, presets). Prepare a small demo video or set of screenshots to illustrate the smooth operation and highlight improvements. This is important for stakeholder review.

Acceptance Criteria: Documentation covers installation, basic usage, and advanced tips (batch usage via CLI, etc.). No outdated references to Streamlit UI except as note for future. The demo (could be informal) shows the app doing a quick adjust and generating a pot in under 2s, etc., demonstrating we hit our smoothness goals.

- **Milestone:** End of Phase 2 corresponds to a **Beta release (v2.0-beta)**. The app at this point should be feature-complete for our requirements (excluding any stretch goals).

Phase 2 Risks: Compare mode UI complexity might cause delays – mitigation: if running behind, deprioritize compare (ship a simpler compare or postpone to minor release). Performance fine-tuning risk: if performance is already good, Numba might be unnecessary; focus on main bottlenecks only (don't over-engineer). We also must watch for VTK rendering issues (some older GPUs might have issues – have a fallback of showing nothing rather than crash).

Phase 3: Scale, Quality & Hardening (Weeks 9–12)

Objective: Final optimizations, cross-platform polish, extensive testing, and preparing for production release.

- **Task 3.1: Parallel Batch & CLI Improvements**

Owner: Core Engineer.

Details: Finalize the `adapters.batch.build_from_yaml` with process pool parallelism. Integrate OMP thread control. Add command-line flags to CLI to utilize multiple cores. Also, ensure the GUI can call this (maybe from a "Run Batch" action if we add it, or just rely on CLI).

Acceptance Criteria: Using a test YAML with e.g. 10 recipes, measure speed with parallel vs sequential. Expect ~ linear speedup up to CPU count. Ensure the manifest JSON is correctly written. If a mesh fails in batch, the others still complete and error recorded. The CLI usage (if any provided like `potfoundry --batch recipes.yaml`) works without crash and outputs summary.

- **Task 3.2: Cross-Platform & Installer**

Owner: Release Engineer.

Details: Test and fix issues on macOS and Linux. On macOS, handle signing and notarization: set up an Apple Developer ID for signing the app bundle, run `codesign` and `notarize`. On Linux, maybe produce an AppImage or just instruct pip usage. Ensure that on Mac, the Qt frameworks are properly included by PyInstaller (they often are).

Acceptance Criteria: The app runs on macOS (tested on at least one Intel and one M1 machine), with the .app bundle double-clickable (no unidentified developer warning after notarization if we go that far). On Linux, verify it runs on a common distro (we might not package extensively for Linux beyond

wheel, but ensure no Linux-specific code issues). Document any platform-specific steps (like needed libraries).

- **Task 3.3: Fuzz Testing & Stability Audits**

Owner: QA Engineer.

Details: Increase fuzz testing range: thousands of random parameter sets overnight test to ensure no crashes or non-manifold outputs. Use a tool or script to simulate heavy usage: e.g. open app, randomly tweak parameters for 5 minutes, see if memory grows (memory leak check) or app crashes. Also test extreme sizes (max sliders) to see if it gracefully handles (maybe just slow but does it complete).

Acceptance Criteria: 0 crashes in fuzz test run of e.g. 1000 random cases (within allowed param ranges). Any found issues are fixed (e.g. maybe found a combo that caused an assertion, fix the logic). Memory usage should remain roughly stable (no large leak; some caching will increase memory but bounded). Tools like Valgrind (for native leaks in VTK) show no major issues (some minor Qt internal leaks might show but we can ignore if from libs and small).

- **Task 3.4: Final Performance Verification**

Owner: Core Engineer & QA.

Details: Confirm we meet performance targets on a variety of hardware: test on a low-end laptop vs a high-end machine to gauge difference. If any performance issues, consider final tweaks (like reduce default `n_theta` on very slow machines if needed). Also ensure the UI is locked at most 60 FPS (avoid hogging CPU to redraw too fast).

Acceptance Criteria: On a reference low-spec (say 4-core, no discrete GPU) machine, preview still under ~500ms for standard model (maybe not 200ms, but acceptable). On a high-end machine, everything is super fluid. Document recommended system requirements (e.g. “Requires OpenGL 3.3 GPU; 8GB RAM for very high quality; tested on X, Y configurations”).

- **Task 3.5: Documentation & Version 2.0 Release**

Owner: Architect and UX.

Details: Finalize all documentation (user guide, API reference for core if we provide it to developers). Write a migration guide for any existing users (though probably not many, given it was prototype). Prepare the official changelog for v2.0. Engage a few beta testers if possible to get feedback and ensure no UX issues remain.

Acceptance Criteria: All issues from beta feedback are resolved or acknowledged. The documentation is sufficient that a new user can get started easily. We publish v2.0.0 on GitHub/website and PyPI. The release is tagged, and binary installers for Win/Mac (and maybe an archive for Linux) are attached. The team agrees that the app is production-ready (smooth, fast, reliable as per mission).

- **Task 3.6: Streamlit Parity (if time permits)**

Owner: Optional, Core Eng.

Details: Revisit the Streamlit prototype and adjust it to use the new core. If some features in desktop are easy to port (like using the same PotParams definitions to create Streamlit sliders), implement them. Ensure that at least the basic functionality still works in Streamlit (so the “future port” path is proven).

Acceptance Criteria: Running `streamlit run app.py` (refactored) brings up the same interactive designer in a browser, using the core. Possibly not all desktop features (like 3D preview) are present,

but user can adjust and download STL. This confirms our core is UI-agnostic. If this is not feasible within 90 days, this can be deferred, as it's future path.

Phase 3 Risks: Notarization on Mac can be time-consuming due to Apple's process – start early. Beta user feedback might request changes – buffer last 2 weeks for any quick UX improvements or bug fixes. Also, heavy testing could uncover a deeper issue (e.g. VTK bug on certain GPU); mitigation: if severe, consider fallback to PyQtGraph for those cases, or clearly document the limitation.

Summary Timeline:

- *Weeks 1–2:* Core refactor, binary STL, initial test passing.
- *Week 3:* Schema v2 + migration done, start basic UI integration.
- *Week 4:* Basic UI working (Phase 1 complete, internal alpha).
- *Weeks 5–6:* Build out full UI, presets.
- *Week 7:* Compare mode, caching and perf optimizations.
- *Week 8:* GUI tests, documentation updates (Phase 2 complete, beta release).
- *Weeks 9–10:* Parallel batch, cross-platform packaging.
- *Weeks 11–12:* Fuzz/stress testing, final polish, release 2.0.

Each phase completion will be a checkpoint to review against acceptance criteria. We will adjust tasks if any phase took longer (some buffer is built in by extending to 90 days).

8) Testing Strategy

A multi-faceted testing strategy will ensure quality and prevent regressions:

Unit and Functional Tests (Core)

- **Geometry Calculations:**
 - Prepare known simple cases: e.g., a pot with height 10, top=bottom= diameter 10, wall 1, drain 0 (basically a cylinder with closed bottom). We know the expected vertex count ($n_{\theta}(n_z+1)^2$ surfaces etc.) and face count. Write a test that `build_mesh` returns those counts and that the mesh is manifold. We can check manifoldness by verifying each edge appears twice or once correctly. For a cylinder with no drain and closed rim, Euler characteristic should be 2 (closed surface) – we can test that as an indicator.
 - Test style functions: maybe skip verifying the math exactly, but ensure that for example `hr_petals=0` yields a symmetrical shape (like no variation), or that increasing petal amp changes the radius array accordingly.
 - Edge case: bottom_thickness equal to height (meaning essentially no inner cavity except maybe just a tiny at rim) – ensure function doesn't break (maybe inner surface ends up just a ring at top).
 - Zero drain vs non-zero drain: verify that when drain=0, the inner bottom surface has no hole (so no boundary at bottom), whereas when drain>0, an inner cylindrical hole is present (which we see as additional faces count).
- **STL Writer:**
 - Already discussed: test with a small mesh (like a single triangle or a tetrahedron shape made of 4 triangles forming a closed pyramid). Write binary STL, then read it (we can parse the binary easily or use `trimesh` library to load and see if it matches).

- Also test ASCII writer similarly (even though it's debug, ensure it outputs expected text).
- Intentionally test a path where output file exists (to ensure our replace logic works). Possibly test writing to a read-only directory (should raise error, which we catch).
- **Schema Validation:**
- Provide a minimal valid v2 YAML as string to `validate_recipe` and ensure it returns a `ValidatedRecipe` with all defaults applied.
- Test a few failure cases: missing 'name' in a recipe, both style and preset given, unknown style name, negative dimension value – each should produce a specific error message or exception type.
- Test migration: feed a v1 dict with a preset and recipe using 'use', verify the output has 'preset' field etc.
- **Adapters (non-UI):**
- We can simulate usage of `PreviewWorker` in a test by calling `request` and then waiting for result. Possibly we modify `PreviewWorker` to allow synchronous operation in test (like an option to not actually thread out). Alternatively, we test underlying logic (like caching function that generates keys).
- Test caching: call `request` twice with same params and ensure second time it returned immediately (maybe we expose a method in `PreviewWorker` for testing that checks if a given key is cached).
- Test batch: we can run `build_from_yaml` on a small config with 2 recipes (maybe one intentionally invalid to test error capture). Verify that manifest lists one success, one error, and that the STL for success exists with correct name.
- **CLI/Smoke Tests:**
- If we have a CLI entry (like `potfoundry-cli`), run it with `--help` to ensure it prints usage. Run a quick generation via CLI (e.g. `potfoundry-cli --quick --out test.stl` with some inline params or a YAML) and verify file appears. These ensure packaging includes the console script, etc.

GUI Tests

- **Launch Test:** Use Qt's `QTest` framework or `qpytest` (if available) to simulate user actions.
- For example, simulate clicking the "Generate STL" without changing anything (should produce an STL file in default location or prompt).
- Simulate moving a slider: `QTest` can send a mouse drag or we can directly set value and emit signal.
- Verify that certain UI elements update: for instance, after moving height slider, check that the `QVTKWidget` has a new mesh. We might not easily introspect the 3D content, but we could use a hook: e.g., our `ViewModel` could emit a signal when mesh is updated, so test connects to that to ensure it was emitted.
- Test cancel: start a generation on a large mesh (maybe we insert a sleep in `build_mesh` for test), then click cancel and verify that `result_ready` is not emitted (or is ignored).
- We'll also do a manual exploratory test session as part of QA: open the app, try all features (preset save, compare mode toggling, resizing window, etc.) to catch any crashes or layout issues.

Golden Master Tests

- We save "golden" outputs for certain known inputs (small ones for practicality):
- E.g., for `PotParams` = (height=100, top_dia=100, bottom_dia=100, wall=3, bottom=3, drain=0, style=none plain, quality 36x18), we generate a mesh and save its vertices & faces or a hash of them. This is our reference. If changes in code cause output differences, the test will flag it. This ensures we don't inadvertently change geometry (unless intended due to bug fix).

- We can't store huge meshes in repo, but a moderate one is fine. Or store just counts and a checksum (like MD5 of sorted vertices).
- For each style, we might have one golden example output (ensuring that style's generation remains consistent across refactors).
- Also golden images: we might render the preview to an image and compare to baseline. But rendering differences on different GPUs make image comparison tricky. Probably skip UI golden images.

Fuzzing and Edge Tests

- As mentioned, we will generate a large number of random parameter sets within allowed ranges and run `build_mesh`. Use Python's hypothesis library or simply random loops.
- We particularly watch for exceptions or very slow cases. If any found, fix or add to known limitations (e.g. if `flare_exponent` extremely high causes oscillation – maybe we restrict range).
- Feed random slight invalid inputs to `validate_recipe` to ensure it always errors out cleanly, not crash.

Continuous Integration Matrix

- We set up CI to run tests on:
 - Windows (latest, Python 3.11 and 3.12).
 - Ubuntu (maybe just one Python version).
 - macOS (one version). The core tests run on all. GUI tests on Linux (using xvfb to emulate display) to avoid flakiness on Windows/mac in CI. We might exclude GUI tests on mac CI because of lack of display, but run them on Linux with virtual display.
- We also will have a nightly or on-demand job to run longer tests (like the heavy fuzz or performance tests) that we might not run on every commit (to not slow CI). Those can run on a schedule.

Test Data and Golden Meshes

- We will create some sample output STLs to use for testing in the repository (small binary STLs to verify reading function and such).
- For performance, we will not fail tests if performance is slightly off (to avoid false fail on CI if machine slow), but we might log performance to ensure no 10x regressions. Perhaps implement a benchmark suite separate from tests.

User Acceptance Testing

- Before final release, have a couple of internal users or friendly beta testers try the app on their own machines and workflows. They will provide feedback which we convert to either bug fixes or improvements (if minor and time allows) or log for future.
- Specifically ask them to note any crashes, confusing UI parts, or if something felt slow.

Overall, our testing strategy covers core correctness, performance, and full integration (GUI) to ensure smooth operation, speed, and reliability, aligning with the mission goals. We will maintain these tests going forward to catch regressions, especially given the deterministic outputs (which makes automated checking feasible).

9) Release Engineering

We treat build and release as first-class concerns to deliver a professional product.

Versioning and Changelog

- Adopt **Semantic Versioning**: starting this new desktop-focused version as **2.0.0** (since presumably 1.x was the Streamlit prototype). Future changes: only backward-compatible enhancements -> minor version bump, breaking changes -> major bump, bug fixes -> patch bump.
- We maintain a `CHANGELOG.md` where each release's changes are logged (features, fixes, etc.). This will be updated in each PR that constitutes a user-visible change.
- The version number is defined in one place (possibly `__version__` in core `init` and used in the UI about dialog and CLI `--version`).

GitHub Actions CI/CD

- **CI Pipelines**: as described, we have workflows for test/lint. Additionally:
 - On pushes to main or PRs: run lint, mypy, tests (fast).
 - Possibly a separate workflow on a nightly schedule for extended tests (fuzz/perf).
- **Build and Deploy**: Set up GitHub Actions to build distributables:
 - We create jobs to run PyInstaller to build the standalone executables for Windows and Mac. This likely means running on a Windows runner and a macOS runner, respectively.
 - Use PyInstaller config: a `.spec` file where we can fine-tune included files. For example, exclude Qt translations, unused image formats, etc., to trim size. Ensure the VTK Python libs are included (PyInstaller should auto-detect imports, but we might have to add data files like VTK's DLLs explicitly).
 - After building, we can use the actions/upload-artifact to store them, or on a tagged release, attach to GitHub release.
 - If code signing certificates are available, use them:
 - Windows: use signtool (but likely we skip code signing in open-source context unless we have a cert).
 - Mac: use the Developer ID certificate stored securely (maybe in GH Secrets) and run codesign. Notarization can be automated by uploading to Apple and polling for result.
 - If we cannot fully automate notarization, we might do it manually for final release.
- Linux: maybe just build a wheel (.whl) for core library distribution via PyPI. For a GUI, an AppImage build could be done but often Linux users can run from source with dependencies installed. We'll document that.
- **Package Size & Optimization**:
 - PyInstaller tends to produce large executables due to bundling Qt libraries. We will:
 - Use UPX compression on binaries to shrink size (PyInstaller can do this by default for suitable parts).
 - Exclude debug symbols and unnecessary Qt plugins (like we only need `platforms/qwindows.dll` on Windows, not all of them).

- Possibly exclude QtWebEngine or QtWebKit if PySide pulled them in accidentally (we should avoid importing those).
 - Verify that only one Qt binding is included (PyInstaller disallows mixing PyQt and PySide, which we won't do).
 - With VTK, try to limit which modules are included. If using PyVista, it may internally import a lot of VTK. We might consider using VTK's modular imports to only bring what we need (likely `vtkCommonCore`, `vtkIOGeometry` for STL reading maybe, `vtkRenderingOpenGL2` etc.). If needed, tweak PyInstaller spec to remove modules not used.
 - Our target is maybe ~100 MB per packaged app. If it's more, it's acceptable but we try to not blow up too much.
- Wheel packaging: produce a `potfoundry-core` wheel that includes core and possibly adapters (since adapters have no heavy deps, just Pydantic, numpy, etc.). The GUI parts (PySide6, VTK) would not be dependencies of core wheel (to keep it lightweight). Alternatively, we provide one `potfoundry` wheel that depends on PySide6 and VTK which are pip-installable (PySide6 is pip-installable, VTK too but large). We might split into two packages for separation. But since mission is desktop app, perhaps one package is fine and users can optionally avoid installing GUI by only using core submodules.
- **Pre-commit Hooks:** We will add a pre-commit config so developers automatically run ruff, black (if we use it), etc., on commit. This keeps code style consistent. Also hook mypy to avoid type regressions.
- Acceptance: All contributors will install pre-commit and we also enforce via CI in case someone bypasses.
- **Issue Tracking & Project Board:** Not exactly release engineering, but we likely maintain a list of issues/risks on GitHub. The risk register (below) could be transformed into GitHub issues to keep track of mitigation tasks.
- **Release Process:**
 - Create a GitHub Release with notes (populated from changelog). Attach built executables.
 - On PyPI, upload the wheel (and maybe source distribution if we want to allow pip install for core).
 - Verify installation: we will test that downloading the exe works on a clean machine, pip install works in a new venv, etc., as part of release checklist.
- **Notarization (macOS):**
 - Apple requires signing and notarizing for the app not to be blocked on default settings. We should do this for Mac release:
 - Use `codesign` with `Developer ID Application` certificate for all binaries in the .app and then `xcrun altool --notarize-app` to send to Apple. They return success/failure (we script and wait for success).

- This requires secrets for Apple ID which can be stored in GitHub Secrets to automate, or we do a local signing then upload.
- If not done, Mac users would have to right-click Open to bypass, which is not ideal for broad distribution.

- **Post-release Telemetry (if any):**

- If telemetry is implemented and user opted in, we might have a server or just rely on user reports. For now, maybe not set up a whole telemetry server due to time, but keep the code ready.

- **Continuous improvement:**

- After 2.0, we'll use CI to also perhaps build nightly dev builds for testers if needed.
- Also set up tools like Sentry (for crash reporting) if we wanted automatic error collection – but only if user permits, likely not for this timeframe.

In summary, release engineering ensures that when we deliver 2.0, it's easy for users to obtain and trust. We leverage CI to automate building and incorporate best practices (signing, versioning, etc.) to give a polished delivery.

10) Risk Register

We identify potential risks and their mitigation strategies:

1. **Large Mesh Memory Exhaustion:** If a user cranks quality to extremely high values (e.g., $n_{\theta}=10000$), memory usage for mesh could be several GB, possibly crashing the app or system.
Mitigation: Impose reasonable limits on sliders (e.g., n_{θ} max ~ 1000 in UI). Also, detect if $n_{\theta} * n_z$ exceeds a threshold (like 10 million) and warn or refuse. Document recommended limits. In batch mode, we can have a safeguard to not spawn too many huge jobs at once.
2. **No GPU / Poor OpenGL Support:** On some systems (old PCs, remote desktop, VMs), OpenGL 3 may not be available, and VTK rendering could fail or be extremely slow (fallback to software).
Mitigation: Do a GPU/GL check at startup (VTK can query renderer info). If insufficient, notify user “3D preview disabled (OpenGL 3+ required)”. Allow the app to run with preview panel blank rather than crashing. Possibly allow switching to a simpler preview (like a 2D outline) in no-GPU environments as a fallback.
3. **VTK Integration and Packaging Issues:** VTK brings many components; PyInstaller might miss some or produce an unreliable exe (e.g., QVTK plugin issues). There's a risk the frozen app fails to initialize the VTK widget due to missing libraries.
Mitigation: Test the frozen build early (in Phase 1/2). Use PyInstaller analysis to add any hidden imports. Consult VTK forums if needed. As backup plan, if VTK proves too problematic in frozen form, pivot to PyQtGraph (which might be easier to package) – but only if absolutely necessary. For now, we anticipate with PyInstaller tricks it's manageable.

4. **Unicode or Unusual File Paths (IO Issues):** Windows has issues if file path lengths > 260 or non-ASCII chars if not handled. Also, user may input non-ASCII in preset names leading to file names.
Mitigation: Use Python's built-in UNC path handling on Windows (prefix `\\?\\` if long paths needed). Ensure we open files with proper encoding (though STL binary isn't text). We test saving to a unicode path (like "C:\测试\output.stl") to ensure it works. If any library (like yaml or PyInstaller) has encoding issues, handle them (all modern libs should support unicode paths).
5. **BLAS Thread Oversubscription:** Using numpy in multiple threads or processes can oversubscribe CPU, hurting performance (especially on machines with many cores, each numpy might use all cores).
Mitigation: As planned, for batch (multiprocess), set `OMP_NUM_THREADS=1` in each worker. For interactive preview (single thread), let numpy use multithreading if it wants – typically for our workload (not heavy linear algebra) numpy might not even multithread trig and small ops. We'll monitor CPU usage; if we see it spawning too many threads, we might globally set a lower thread count. We'll mention in docs how to adjust if needed.
6. **Multithreaded UI Race Conditions:** Complexity of signals and threads could cause race issues, e.g., a result comes just as user closes the app, possibly accessing deleted objects.
Mitigation: Use Qt's thread-safe signal mechanism which queues to main thread, reducing direct race. In ViewModel, keep track of app state: if app is closing, drop incoming results. Also, ensure to `disconnect` signals in object destructors. Test with closing window during heavy operation to see if any crashes occur (fix by checking pointers or using guards like if thread still running on close, wait/join it with a short timeout).
7. **Differing Floating-Point Results (Non-determinism):** On different CPU architectures (x86 vs ARM) or with different math libraries, the generated mesh might differ by tiny amounts (rounding). This could cause golden test mismatches or user confusion if STLs differ by machine.
Mitigation: Differences should be minimal (floating error $\sim 1e-9$). Not a big user issue, but for tests, we avoid exact float comparison – use tolerance. And for determinism, ensure using the same algorithm path on all (numpy usually consistent). Document that outputs may vary in vertex order if any parallelization is introduced (but we try to keep consistent ordering).
8. **Backward Compatibility of YAML:** Users of v1 might have custom scripts or expectations. With v2 changes, if we drop or rename fields, they might be confused.
Mitigation: Provide the migration and print a log message, "Config v1 detected, automatically converted to v2". Also supply a small script or function to upgrade files for users. Keep support for reading v1 for at least one major version with warnings, so no one is stuck.
9. **User Error Handling & Frustration:** If something fails (like invalid params or file save error), if not communicated well, user may get frustrated or do incorrect things.
Mitigation: Make error messages very clear and actionable. E.g., if "Drain hole too large" error, message should guide "Increase bottom diameter or reduce drain size." If file save fails "Disk full", mention checking disk space or permissions. We will also add an "Error Log" panel (maybe in Help menu) that can show technical details for support, if needed.
10. **Performance on very old hardware:** The 200ms preview target might not hold on a 10-year-old PC or with huge designs.

Mitigation: Provide quality presets and default to a reasonable preview resolution that most machines handle. The user can still set high quality for final export, but for preview, we manage expectations. Also, if detect >200ms generation consistently, maybe auto-switch to draft preview mode (as mentioned incremental refinement). Essentially degrade gracefully on lower-end systems.

11. **Concurrent Batch and UI usage:** If the user somehow runs a batch generation while also editing in UI (maybe via GUI batch or separate thread), it could conflict (like both trying to use core at same time in same process, though core functions are pure and stateless so likely fine).

Mitigation: If we implement batch in UI, consider running it in a separate process or at least separate threads that do not interfere with preview thread. Also disable interactive editing while batch running (or very least, warn that performance might drop). This risk is minor.

12. **Qt or VTK Version Updates:** Over the project, new PySide or VTK versions may release (especially as Qt6 evolves). There's a risk an update breaks our app (e.g. behavior change in QThread or a VTK bug).

Mitigation: Pin versions in requirements for now (like PySide6==6.5.*, VTK matching what we test). We will regularly test with updates in CI and adjust code if needed. Also, not upgrade in the middle of development unless needed.

13. **Security of distributed binaries:** We need to ensure our distributed app doesn't accidentally include sensitive info or that it's not tampered.

Mitigation: Use code signing (already planned). Also publish checksum of the installer on our site so advanced users can verify. Keep our build pipeline secure (GH Actions secrets safe, etc.).

14. **Misuse or extreme scenarios not accounted for:** Users might try weird stuff, like extremely large values (maybe specifying a pot 10000 mm tall). Our math might handle it but could also lead to huge coordinates causing visual clipping or precision issues.

Mitigation: Put some reasonable limits in UI (e.g., max height 1000 mm since printers rarely exceed that). Document that beyond certain size, user might need to adjust system (like VTK camera far clip if huge scale, but unlikely needed). Ensuring floats (32-bit) might lose precision at extremely large coordinates, but that's okay – 0.01 mm precision is enough.

Each risk will be tracked as an issue and we will verify mitigations during development. None of these are show-stoppers given our plan, but awareness and planning ensure they don't surprise us in final stages.

11) First 3 PRs (ready to open)

We outline the initial three pull requests to kick off implementation, with details and acceptance criteria for each:

PR #1: Implement Binary STL Writer & Atomic Save (core.io.stl)

Summary: Introduce the new binary STL export function and update the system to use it by default, including tests and benchmarks.

- **Changeset:**

- Add `core/io/stl.py` with `write_stl_binary` and `write_stl_ascii` as specified. Include docstrings and logging.
- Modify any place currently calling `write_ascii_stl` (e.g., in CLI or Streamlit code) to call `write_stl_binary` instead, unless in debug mode.
- Implement atomic temp file logic in `write_stl_binary`.
- Add `core/io/tests/test_stl_io.py`:
 - Test writing a small mesh and reading it back (we can read binary by struct or use `numpy.fromfile`).
 - Test that header is 80 bytes, count correct, etc.
 - Test ascii writer similarly (perhaps compare the content strings).
- Add a simple benchmark script `core/bench/bench_stl_write.py` (not run in CI by default) to generate a random large mesh and time binary vs ascii. Not part of tests, but as documentation for maintainers. Include results as comments or in README.
- Ensure `write_stl_ascii` is marked for debug use (maybe emit a warning if used).
- Update CLI `build_from_yaml` to use binary by default when writing each file (except maybe if user passes a flag `--ascii`).
- **Validation:** All new tests must pass. The ASCII vs binary size/time differences should align with expectations (we might print in test logs or ensure at least that binary file < ascii file).
- **Benchmark Data (to include in PR description):**
 - e.g., "On 100k triangles: binary write 0.1s, 5.0 MB; ASCII write 2.0s, 24.8 MB – about 20× faster, 5× smaller, consistent with sources."
- **Documentation:** Update any user docs or help text to say STL export is now binary. Mention if any minor differences (like some old workflow expecting ASCII, but we consider that debug now).
- **Acceptance Checklist:**
 - [] Saving an STL through the code yields a binary file that MeshLab/other CAD can open without issues.
 - [] Partial write simulation: if we terminate the program mid-write (simulate by monkeypatching an early return), the target file is either intact old version or not created – never half-written.
 - [] The code passes on Windows, Linux, Mac (especially ensure file replace logic works on Windows which doesn't allow atomic rename over existing without permission – we use `os.replace` to handle).
 - [] All tests for STL I/O pass.

(This PR establishes the "Reliability (crash-safe IO)" and "Speed (binary output)" improvements from the plan.)

PR #2: MVVM Refactor, Preview Worker & Non-Blocking UI

Summary: Major refactor to implement the core/UI separation and the threading for preview generation. This sets the stage for the desktop app smoothness.

- **Changeset:**
 - Restructure project into modules: move geometry, io, schema into `potfoundry.core`; define `PotParams` and other models (could use Pydantic or dataclass).

- Implement `PotViewModel` (in `potfoundry.adapters`):
 - It should hold current `PotParams` and perhaps a current `MeshQuality`.
 - Have methods `generate_preview` and signals `preview_ready`, `progress`, `error`.
 - Inside, create `PreviewWorker` (QRunnable-based). Actually, to simplify, `ViewModel` can manage a `QThreadPool` and static functions to run in thread.
 - For now, it can simply call `build_mesh` in a thread and emit result. (Detailed caching can come in later commit if needed).
 - Provide a `cancel_preview` method that sets a cancel flag and/or ignores the next result.
- Basic Qt Desktop UI scaffold:
 - Create `main.py` or similar that sets up `QApplication`, constructs the `PotViewModel`, and a `MainWindow` with few controls and a `QVTKOpenGLNativeWidget`.
 - Connect a couple of signals: e.g., a slider for height -> on value change, update `viewModel.params.height` and call `viewModel.generate_preview()`.
 - On `preview_ready`, call a method to render the mesh in VTK (for now, just use `PyVista`: convert vertices/faces to `PolyData`, add to plotter in the VTK widget).
 - Make sure the UI doesn't freeze: test by generating a deliberately heavy mesh and confirm one can still interact with window.
- Ensure that the Streamlit code (if present) can still use core (we might not fully integrate now, but at least ensure import paths work).
- Unit tests:
 - We can simulate a `PreviewWorker` outside GUI by invoking it and waiting for result (maybe need to run Qt event loop in test, which is tricky). Alternatively, test the core of it: that calling `build_mesh` in a separate thread yields correct result and that cancel flag logic works.
 - Might skip heavy threading tests in CI if not stable, but we can test small usage.
- Basic cache mechanism (optional in this PR or next): maybe include at least caching theta arrays as a demonstration. Write small test for cache: call internal function twice and ensure no difference and maybe time difference if measurable.
- **Acceptance Checklist:**
 - [] Project imports are clean: `import potfoundry.core.geometry` works, etc. (Refactor doesn't break things).
 - [] We can run `python -m potfoundry.ui.desktop.main` and a window appears with at least one slider and a basic 3D view, and adjusting it updates the model.
 - [] UI remains responsive: verified by the `RealPython` example (click counter continues to update or the window can be moved while generating).
 - [] No core logic changes have broken core tests (all core tests still pass).
 - [] New architecture documentation: update README to describe how core vs adapters vs UI are structured.
 - [] (Later, when full UI is done, we'll have more checks, but in this PR, focus is the structure and non-blocking threading.)

(This PR covers multiple decisions: "UI thread never blocks", "MVVM separation", and sets groundwork for "Desktop-first architecture". It's a large PR, so may be broken into smaller ones, but we combine for overview here.)

PR #3: Schema v2, Preset Manager, and Streamlit Compatibility

Summary: Introduce the new YAML schema version with migration, and add preset management in the UI. Also ensure we maintain a path for Streamlit (the “UI-agnostic core” demonstration).

- **Changeset:**

- Add `core/schema.py` with `validate_recipe` and `migrate_v1_to_v2`. Use Pydantic models for Config if convenient, or do manual validation as per spec.

- Write tests for schema: e.g., one for migrating an example v1 (like what's in `user_files` snippet) to v2 and asserting certain keys present.

- Integrate migration in CLI or where config is loaded.

- **Preset management:**

- Decide where to store user presets. Possibly in a YAML in user's home directory (like `~/.potfoundry-presets.yaml`).
- Implement functions to load/save presets from that file.
- In UI, add a Preset dropdown. Populate it with any presets (including built-ins delivered with app, which could be loaded from a packaged YAML).
- Add “Save Preset” action: takes current PotParams, asks for name, and writes to presets file, then refresh dropdown.
- Add “Delete Preset” (optional, not asked but useful) perhaps via context menu on dropdown.
- Under the hood, presets are basically storing a PotParams (could reuse schema structure or simply storing param values).
- Ensuring no heavy GUI deps in core: preset save/loading can be done in adapters or a small separate module. It's mostly file IO and updating UI.

- **Streamlit bridge:**

- Possibly reintroduce a `potfoundry.ui.streamlit` module that contains functions to render controls using streamlit and call core. (However, fully implementing that might be beyond scope of PR; maybe just ensure core usage is shown to work in streamlit environment.)
- Alternatively, at least run a quick manual test: write a small streamlit script using the new PotParams and `build_mesh` to verify no Qt needed. This is more of documentation.

- **Documentation:** Update the YAML format section of README. Provide an example v2 YAML and how to migrate from v1. Also mention that core now uses pydantic (if so) and how that might affect usage.

- **Tests:**

- For presets: simulate saving a preset and loading it. Perhaps call the underlying functions directly (without UI). Make sure round-trip yields same params.
- For Streamlit, we might just have a basic test that ensures `build_mesh` can be called in a plain Python context (which is already done in core tests). If we had any dependency on Qt in core, this test would catch it (e.g., try import `potfoundry.core` in a subprocess with no PySide installed, if we declared PySide optional).

- **Acceptance Checklist:**

- [] A v1 YAML (maybe from prototype) can be loaded through our new code and results in the same number of pots generated as before (we could test with a small example).

- [] Presets UI: Create a new preset via UI, then restart app (simulate by re-loading presets file) and see the preset persists.
- [] No core functionality is tied to Qt: test by running a headless script that calls `build_mesh` (shouldn't require a `QApplication`, etc.). If someone only installs core (without `PySide6`), importing `potfoundry.core` and using it should work (we might separate extras in setup).
- [] The application still reads old config for batch (if user selects an old YAML, it migrates seamlessly).
- [] Streamlit usage: If possible, run the streamlit app (adjusted to new import paths) to confirm it still basically works. (We might automate that via launching streamlit with a sample yaml input in CI, or just manual for now.)
- [] All schema tests and any new preset tests pass.

(This PR delivers the "Schema & validation v2" decision, and the Preset manager portion of the UI. It also ties up the requirement that core is UI-agnostic by demonstrating Streamlit compatibility.)

These first three PRs implement foundational improvements: faster STL output, a robust threaded architecture, and structured config handling. Each PR is focused and includes tests and documentation as appropriate. By merging these, we lay the groundwork for subsequent PRs focusing on performance tuning, UX polish (like compare view, design health), and final release prep. Each PR is sized to be reviewable (though PR#2 is quite large, it could be split into "Refactor core + basic ViewModel" and "Threaded preview + basic UI" if needed).

In all PRs, we will include acceptance criteria checklists (as above) in the description to track completion. Code reviews will verify that criteria are met and that no part of the mission (smoothness, speed, reliability, etc.) is regressed by the changes. After these three PRs, the project should already show clear improvements in operation and maintainability, aligning with our ultimate goal for PotFoundry.
